Google Maps – System Design & Architecture

বাংলা Presentation Speech ও Learning Guide (Software.pptx-এর ২৩টি স্লাইডের পূর্ণ বিশ্লেষণ)

এই গাইড কীভাবে ব্যবহার করবেন

প্রতিটি স্লাইডের জন্য একই কাঠামো আছে: **A.** পরিচিতি, **B.** ব্যাখ্যা, **C.** Presentation Speech, **D.** Additional Knowledge, **E.** শিক্ষকের সম্ভাব্য প্রশ্ন, **F.** Technical Clarification, **G.** Delivery Tips। প্রথমে B পড়ে বিষয়টি বুঝুন, তারপর C দেখে জোরে জোরে প্র্যাকটিস করুন। D অংশটি স্লাইডের বাইরের প্রশ্নের জন্য। সবশেষে (স্লাইড ২৩-এর পর) Final Preparation Section আছে।

⚠️ শুরুতেই জানা দরকার: PPTX-এর গুরুত্বপূর্ণ সতর্কতা

স্লাইডগুলো পড়তে গিয়ে কিছু বিষয় পেয়েছি যা শিক্ষকের চোখে পড়তে পারে। আগে থেকে জানা থাকলে আপনি আত্মবিশ্বাসের সঙ্গে সামলাতে পারবেন।

1. **এটি Conceptual/Reference Architecture।** Google তাদের Maps-এর ভেতরের (proprietary) architecture প্রকাশ করেনি। স্লাইড ২-এর Notes-এও লেখা আছে এটি reference architecture। তাই presentation-এর শুরুতেই বলে দিন: "আমরা Google Maps-এর সম্ভাব্য ও প্রচলিত design কীভাবে হতে পারে তা দেখাচ্ছি, Google-এর সত্যিকারের internal design নয়।"
2. **স্লাইড ৩, ৪, ১৯ ও ২২-এর Speaker Notes ভুল।** এই চারটি স্লাইডের Notes হুবহু একই লেখা, যা আসলে পুরো presentation-এর সমাপনী সারাংশ ("Finally, this slide brings together all the major components..."। স্লাইডের বিষয়ের সঙ্গে এর মিল নেই। **এই Notes দেখে পড়বেন না।** এই গাইডের Speech ব্যবহার করুন।
3. **স্লাইড ১৭-এ ভুল শ্রেণিবিভাগ।** "Distributed Cache (e.g., Spanner, Bigtable with cache)" লেখা আছে। কিন্তু Spanner ও Bigtable হলো **database/storage system**, cache নয়। Cache হিসেবে Memcached/Redis ব্যবহার হয়।
4. **CDN-এর অসঙ্গতি।** স্লাইড ৭-এ "CDN (Cloudflare)" ও "Mapbox / Google Maps Tiles" আছে, আর স্লাইড ১৭-এ "Google Cloud CDN"। Google নিজের Maps-এর জন্য Cloudflare ব্যবহার করবে, এটা বিশ্বাসযোগ্য নয়। এগুলোকে "উদাহরণ হিসেবে প্রচলিত technology" বলুন।
5. **নির্দিষ্ট technology-গুলো উদাহরণমাত্র।** PostgreSQL+PostGIS, Redis, H3 (এটি আসলে Uber-এর তৈরি), Geohash, ও "Google Cloud API Gateway" (এটি Google Cloud-এর গ্রাহকদের জন্য একটি product) হলো সাধারণ geospatial system-এ ব্যবহৃত পরিচিত technology। Google Maps এগুলোই ব্যবহার করে, এমন নিশ্চিত প্রমাণ নেই।
6. **স্লাইড ৪-এর সংখ্যাগুলো (<100 ms, 99.99%)** Google-এর ঘোষিত SLA নয়, এগুলো একটি ভালো design-এর লক্ষ্যমাত্রা (target)।

7. **স্লাইডের ক্রমে ছোট অসঙ্গতি।** স্লাইড ১৯ (Bottlenecks) মাঝখানে বসেছে, আর স্লাইড ২০-২১ (Data Pipeline, AI) তারপর এসেছে। স্লাইড ২-এর Notes-এর শেষ লাইন সরাসরি Geospatial Database-এর কথা বলে, অথচ মাঝে স্লাইড ৩-৪ আছে। Transition বলার সময় এই গাইডের বাক্যগুলো ব্যবহার করুন, Notes-এরগুলো নয়।
8. **স্লাইড ২২-এর Reference [3], [4], [5], [8]** (Contraction Hierarchies, A*, Route Planning, Tail at Scale) স্লাইডের মূল অংশে সরাসরি আসেনি। শিক্ষক জিজ্ঞেস করলে এগুলো কোন বিষয়ের সঙ্গে সম্পর্কিত তা বলতে পারবেন (স্লাইড ২২-এ দেখুন)।

স্লাইড ১: Title / Welcome

A. Slide Identification

- **Slide Number:** ১
- **Slide Title:** "Inside Google Maps: how a global mapping platform is designed"
- **উদ্দেশ্য:** Presentation-এর বিষয় ও দলের সদস্যদের পরিচয় দেওয়া, এবং শ্রোতাদের জন্য আগ্রহ তৈরি করা।

B. ব্যাখ্যা

এই স্লাইডে পাঁচজন সদস্যের নাম ও ID আছে: Mutiur Rahman (2207097), Alok Baul (Turjo) (2207098), Aavas Roy (2207099), Plabon Barua (2207101) এবং Md. Sabbir Hossain (2207102)। শিরোনামের মূল শব্দ হলো "global mapping platform", অর্থাৎ Google Maps কেবল একটি অ্যাপ নয়, এটি পুরো পৃথিবীর জন্য তৈরি একটি বিশাল প্ল্যাটফর্ম। "How it is designed" মানে আমরা এর **System Design** দেখব: কোন অংশ কী কাজ করে, অংশগুলো কীভাবে একে অপরের সঙ্গে যুক্ত, এবং কোটি কোটি ব্যবহারকারীকে কীভাবে সামলানো হয়।

System Design কী? একটি বড় software system-কে কীভাবে ছোট ছোট অংশে ভাগ করা হবে, প্রতিটি অংশ কোন technology দিয়ে হবে এবং সেগুলো কীভাবে একসঙ্গে কাজ করবে, তার পরিকল্পনা। বাড়ি বানানোর আগে যেমন নকশা (blueprint) আঁকা হয়, এটি তেমনই।

C. Presentation Speech (প্রায় ১ মিনিট)

> আসসালামু আলাইকুম স্যার এবং আমার সহপাঠী বন্ধুরা। আমাদের আজকের presentation-এর বিষয় হলো "Inside Google Maps: how a global mapping platform is designed"। আমি আপনাদের সামনে আমাদের দলের পক্ষ থেকে উপস্থিত হয়েছি। আমাদের দলে আছেন মুতিউর রহমান, আলোক বাউল (তুর্জ), আবাস রায়, প্লবন বড়ুয়া এবং মোঃ সাব্বির হোসেন।
>
> আমরা প্রায় সবাই প্রতিদিন Google Maps ব্যবহার করি। কোথাও যেতে হলে, রেস্তোরাঁ খুঁজতে হলে, বা রাস্তায় জ্যাম আছে কিনা দেখতে হলে আমরা প্রথমে Maps খুলি। কিন্তু কখনো কি ভেবেছি, ফোনের এই ছোট স্ক্রিনের পেছনে কী ঘটছে? কীভাবে কয়েক সেকেন্ডের মধ্যে পুরো পৃথিবীর মানচিত্র আমাদের হাতে চলে আসে?
>
> আজ আমরা সেটাই দেখব। একটা কথা শুরুতেই বলে রাখি: Google তাদের ভেতরের architecture সম্পূর্ণ প্রকাশ করেনি। তাই আমরা যা দেখাব, সেটা একটি conceptual বা reference architecture, যা প্রকাশিত গবেষণা ও প্রচলিত system design নীতির ওপর ভিত্তি করে তৈরি। চলুন, শুরু করি।

D. Additional Knowledge

এই স্লাইডে আলাদা technical topic নেই। তবে শিক্ষক জিজ্ঞেস করতে পারেন: "তোমরা এই বিষয় কেন বেছে নিলে?" উত্তর: Google Maps-এ Database, Algorithm, Machine Learning, Networking ও Cloud, প্রায় সব CSE বিষয়ের প্রয়োগ একসঙ্গে দেখা যায়, তাই এটি শেখার জন্য চমৎকার উদাহরণ।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **System Design বলতে কী বোঝো?** একটি বড় system-এর অংশ, technology ও তাদের যোগাযোগের পরিকল্পনা, যাতে system দ্রুত, নির্ভরযোগ্য ও scalable হয়।
2. **তোমরা কি Google-এর আসল architecture দেখাচ্ছ?** না স্যার। এটি reference architecture, কারণ Google-এর internal design public নয়।
3. **এই বিষয় কেন বেছে নিলে?** কারণ এটি আমাদের দৈনন্দিন জীবনের সঙ্গে জড়িত এবং অনেক CSE concept একসঙ্গে দেখা যায়।

F. Technical Clarification

স্লাইডে কোনো technical ভুল নেই। তবে "Welcome" লেখাটি শিরোনামের সঙ্গে একটু আলাদা মনে হয়, ঠিক আছে, এটি শুধু অভ্যর্থনা।

G. Delivery Tips

- হাসিমুখে ও ধীরে শুরু করুন। প্রথম বাক্যেই শিক্ষকের দিকে তাকান।
- সদস্যদের নাম বলার সময় স্লাইডে তাদের নামের দিকে হাত দিয়ে ইঙ্গিত করুন।

- "আমরা প্রায় সবাই Google Maps ব্যবহার করি" বাক্যের পর একটু থামুন, এতে শ্রোতারা বিষয়ের সঙ্গে নিজেদের মেলাবে।
- **Disclaimer (reference architecture)** অবশ্যই বলুন, এটি আপনাকে পরে প্রশ্নের চাপ থেকে বাঁচাবে।

স্লাইড ২: System Overview & High-Level Architecture

A. Slide Identification

- **Slide Number:** ২
- **Slide Title:** System Overview & High-Level Architecture
- **উদ্দেশ্য:** Google Maps কী এবং এর পুরো system কয়টি প্রধান স্তর (layer) দিয়ে তৈরি, তার একটি সামগ্রিক ছবি দেওয়া।

B. ব্যাখ্যা

স্লাইডটিকে ওপর থেকে নিচে একটি স্তরবিন্যাস হিসেবে বুঝুন:

ধাপ ১: Client Applications (Web · Android · iOS): ব্যবহারকারী যে অ্যাপ বা ওয়েবসাইট ব্যবহার করে। এখান থেকেই সব request শুরু হয়।

ধাপ ২: API Gateway & Edge: সব request প্রথমে এখানে আসে। এটি চারটি কাজ করে: **Authentication** (তুমি কে?), **Routing** (request কোন service-এ যাবে?), **Rate Limiting** (কেউ অতিরিক্ত request পাঠিয়ে system ভারী করছে কিনা), **Caching** (আগের উত্তর মনে রেখে দ্রুত দেওয়া)।

ধাপ ৩: চারটি Service Domain:

- **Geospatial Services** (Maps · Places · Geocoding): মানচিত্র, রাস্তা, জায়গার তথ্য ও ভৌগোলিক query।
- **Intelligence Services** (Routing · Traffic · ETA): রুট হিসাব, জ্যাম বিশ্লেষণ ও পৌঁছানোর সময়ের অনুমান।
- **Real-Time Processing** (Location · Events · Updates): চলমান লোকেশন, দুর্ঘটনা ও জ্যামের সংকেত প্রক্রিয়া করা।
- **Data Infrastructure** (Databases · Search · Storage): সবার নিচে, যেখানে সব তথ্য জমা থাকে।

Key Capabilities: Maps & Navigation, Search & Places, Real-Time Traffic, Location Sharing, ইত্যাদি।

বাস্তব উদাহরণ: ধরুন আপনি বলছেন "খুলনা থেকে ঢাকা যাব"। আপনার ফোন (Client) request পাঠায় → API Gateway যাচাই করে → Routing Service রুট বের করে → Traffic Service জ্যামের তথ্য যোগ করে → Database থেকে রাস্তার তথ্য আসে → উত্তর আপনার ফোনে ফিরে যায়।

C. Presentation Speech (প্রায় ২ মিনিট)

> এই স্লাইডে আমরা Google Maps-এর পুরো চিত্রটা ওপর থেকে দেখব, যাকে বলে High-Level Architecture। Google Maps আসলে শুধু একটা মানচিত্র অ্যাপ নয়। এটি একটি বিশাল distributed platform, যা মানচিত্র দেখানো, জায়গা খোঁজা, রুট বের করা এবং real-time traffic জানানো, সবকিছু একসঙ্গে করে।
>
> একদম ওপরে আছে Client Application, মানে আমাদের ফোনের অ্যাপ বা ওয়েব ব্রাউজার। আমরা যখন কিছু খুঁজি, সেই request প্রথমে যায় API Gateway ও Edge layer-এ। এটাকে ভাবুন একটি অফিসের রিসেপশনের মতো। রিসেপশন দেখে আপনি কে, আপনার প্রবেশের অনুমতি আছে কিনা, এবং আপনাকে কোন বিভাগে পাঠাতে হবে। Gateway-ও ঠিক সেটাই করে: authentication, routing, rate limiting আর caching।
>
> এর পেছনে system-কে চারটি প্রধান ভাগে ভাগ করা হয়েছে। প্রথমটি Geospatial Services, যা মানচিত্র, রাস্তা আর জায়গার তথ্য সামলায়। দ্বিতীয়টি Intelligence Services, যা রুট হিসাব করে, জ্যাম বিশ্লেষণ করে আর ETA, মানে পৌঁছানোর আনুমানিক সময় বলে। তৃতীয়টি Real-Time Processing, যা চলমান লোকেশন আর দুর্ঘটনার মতো live তথ্য প্রক্রিয়া করে। আর সবার নিচে আছে Data Infrastructure, যেখানে database, cache আর storage থাকে।
>
> এই সবকিছু মিলিয়েই আমরা কম সময়ে, নির্ভরযোগ্যভাবে সারা পৃথিবীর মানুষকে সেবা দিতে পারি। আবারও মনে করিয়ে দিই, এটি একটি conceptual architecture, Google-এর নিজস্ব internal design নয়।
>
> এই পুরো system-এর কাজ বোঝার আগে আমরা দেখব, এটি আসলে কোন সমস্যা সমাধান করতে চায় এবং তার কাছ থেকে আমরা কী কী আশা করি। চলুন পরের স্লাইডে যাই।

D. Additional Knowledge

Distributed System কী? একটি কাজ অনেকগুলো কম্পিউটারে ভাগ করে করা, যাতে একটি নষ্ট হলেও বাকিরা চালিয়ে যায় এবং কাজের চাপ ভাগ হয়ে যায়। উদাহরণ: একটি বড় রেস্তোরাঁয় একজন বাবুর্চি না থেকে অনেকজন থাকে, কেউ ভাত, কেউ মাছ রান্না করে।

Geospatial কী? Geo = ভূমি/পৃথিবী, spatial = স্থান সংক্রান্ত। যে তথ্যের সঙ্গে ভৌগোলিক অবস্থান (latitude, longitude) জড়িত, সেটাই geospatial data।

API Gateway ও Microservices সম্পর্কে বিস্তারিত স্লাইড ১৬-এর অংশে আছে।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **API Gateway কেন দরকার?** সব request-এর জন্য একটি একক প্রবেশদ্বার থাকলে নিরাপত্তা, যাচাই ও নিয়ন্ত্রণ এক জায়গা থেকে করা যায়।

2. **System-কে কেন চারটি ভাগে ভাগ করা হয়েছে?** প্রতিটি ভাগের চাহিদা আলাদা। যেমন real-time অংশে গতি দরকার, আর data অংশে স্থায়িত্ব। আলাদা থাকলে আলাদাভাবে scale করা যায়।
3. **Real-Time Processing ও Intelligence Services-এর পার্থক্য কী?** Real-time অংশ ঘটনা সংগ্রহ ও প্রক্রিয়া করে (যেমন লোকেশন আপডেট), আর Intelligence অংশ সেই তথ্য ব্যবহার করে সিদ্ধান্ত নেয় (যেমন রুট ও ETA)।
4. **এই design-এর Bottleneck কোথায়?** API Gateway যদি সঠিকভাবে scale না করে, সেটিই একক বাধা হতে পারে। এজন্য Gateway একাধিক instance-এ চালানো হয়।

F. Technical Clarification

- স্লাইডটি নিজেই (Notes অনুযায়ী) বলছে এটি conceptual reference architecture। এটি সঠিক ও সৎ উপস্থাপনা।
- "Google Cloud logo" ব্যবহার করা হয়েছে। এটি বোঝায় না যে Google Maps ঠিক Google Cloud-এর public product দিয়েই চলে। Google-এর নিজস্ব অভ্যন্তরীণ infrastructure আছে, Google Cloud তার একটি বাহ্যিক রূপ।

G. Delivery Tips

- **গুরুত্ব দিন:** চার স্তরের ধারণা (Client → Gateway → চার Service Domain → Data)।
- **ইঙ্গিত:** স্লাইডের ওপর থেকে নিচে আঙুল বা লেজার পয়েন্টার চালিয়ে একটি request-এর যাত্রা দেখান।
- **থামুন:** "রিসেপশনের মতো" উপমা বলার পর এক সেকেন্ড থামুন।
- **শেষ:** "এই কারণেই এটি শুধু অ্যাপ নয়, একটি বিশাল system" বলে আত্মবিশ্বাসের সঙ্গে শেষ করুন।

স্লাইড ৩: Problem & Functional Requirements

A. Slide Identification

- **Slide Number:** ৩
- **Slide Title:** Problem & Functional Requirements
- **উদ্দেশ্য:** system কোন সমস্যার সমাধান করে, কারা ব্যবহার করে এবং system-কে কী কী কাজ করতে হবে তা স্পষ্ট করা।

B. ব্যাখ্যা

স্লাইডে তিনটি অংশ আছে।

১. সমস্যা (The Problem): শত কোটি মানুষ ঠিকানা খুঁজতে, দিক জানতে এবং live traffic দেখতে চায়, তা-ও যেকোনো device-এ। আবার পৃথিবী প্রতিনিয়ত বদলায়: রাস্তা বন্ধ হয়, দোকান স্থান বদলায়, ট্রাফিক মিনিটে মিনিটে পাল্টায়। তাই উত্তর হতে হবে ঝটপট ও সব জায়গায় একরকম নির্ভরযোগ্য।

২. Functional Requirements: system-কে যা করতে হবে:

- মানচিত্র দেখানো, pan (টেনে সরানো) ও zoom করা
- জায়গা ও ঠিকানা search করা
- Turn-by-turn navigation সহ রুট দেওয়া
- Live traffic ও সঠিক ETA দেখানো
- Offline map সংরক্ষণ করে ব্যবহার করা
- Live location শেয়ার করা

৩. ব্যবহারকারী: চালক ও যাত্রী, পথচারী ও সাইকেল চালক, API ব্যবহারকারী ব্যবসা ও developer, এবং Android, iOS ও web client।

Functional Requirement কী? system **কী করবে** তার তালিকা। বিপরীতে, **Non-Functional Requirement** বলে system **কতটা ভালোভাবে** করবে (গতি, নির্ভরযোগ্যতা ইত্যাদি), যা পরের স্লাইডে আসছে। উদাহরণ: "ATM থেকে টাকা তোলা যাবে" functional। "তোলা ২ সেকেন্ডের মধ্যে হবে এবং নিরাপদ হবে" non-functional।

C. Presentation Speech (প্রায় ১ মিনিট ৩০ সেকেন্ড)

> যেকোনো system design শুরু হয় একটি প্রশ্ন দিয়ে: আমরা কোন সমস্যার সমাধান করছি? Google Maps-এর ক্ষেত্রে সমস্যাটা হলো, শত কোটি মানুষ যেকোনো ফোনে কোনো জায়গা খুঁজতে, রাস্তা চিনতে আর live traffic দেখতে চায়। আর সমস্যাটা কঠিন হওয়ার কারণ হলো, পৃথিবী কখনো স্থির থাকে না। আজ একটা রাস্তা বন্ধ হচ্ছে, কাল একটা দোকান উঠে যাচ্ছে, আর জ্যাম তো প্রতি মিনিটেই বদলায়। অথচ ব্যবহারকারী চায় উত্তরটা যেন সঙ্গে সঙ্গে পাওয়া যায়।
>
> এই সমস্যা সমাধানের জন্য system-কে অন্তত ছয়টি কাজ করতে হয়। মানচিত্র দেখানো এবং তাকে zoom ও pan করা। জায়গা বা ঠিকানা search করা। Turn-by-turn navigation-সহ রুট দেওয়া। Live traffic ও সঠিক ETA দেখানো। ইন্টারনেট না থাকলে offline map ব্যবহার করা। এবং বন্ধুর সঙ্গে live location শেয়ার করা। এগুলোকে আমরা বলি Functional Requirements, অর্থাৎ system কী করবে।
>
> ব্যবহারকারীও কিন্তু অনেক ধরনের: গাড়ির চালক, পথচারী, সাইকেল আরোহী, এমনকি যেসব ব্যবসা বা developer API ব্যবহার করে নিজেদের অ্যাপে map বসায়। সবাইকে Android, iOS ও web, তিন জায়গায় একই মানের সেবা দিতে হয়।
>
> কিন্তু শুধু "কী করবে" বললেই হয় না, "কতটা ভালোভাবে করবে" সেটাও ঠিক করা দরকার। সেই বিষয়টি নিয়েই আসছে আমাদের পরের স্লাইড।

D. Additional Knowledge

Pan ও Zoom কীভাবে সম্ভব? মানচিত্র ছোট ছোট টুকরো (tile) দিয়ে তৈরি। pan করলে পাশের tile আনা হয়, zoom করলে বিস্তারিত স্তরের tile আনা হয়। বিস্তারিত স্লাইড ৭-এ।

API ব্যবহারকারী ব্যবসা: Uber বা Foodpanda-এর মতো অ্যাপ নিজেদের map তৈরি না করে Google Maps Platform-এর API ব্যবহার করে। এজন্য system-কে বাইরের developer-দের জন্যও স্থিতিশীল রাখতে হয়।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **Functional ও Non-Functional Requirement-এর পার্থক্য কী?** Functional হলো system-এর কাজ (কী করে), Non-functional হলো কাজের মান (কত দ্রুত, কত নিরাপদ, কত নির্ভরযোগ্য)।
2. **Offline map কেন দরকার?** কারণ সব জায়গায় ইন্টারনেট থাকে না, যেমন গ্রামাঞ্চল বা টানেল।
3. **Requirements আগে ঠিক করা কেন গুরুত্বপূর্ণ?** কারণ requirements অনুযায়ীই technology ও design বেছে নেওয়া হয়। শুরুতে ভুল হলে পুরো design ভুল পথে যায়।
4. **তালিকায় নেই এমন কোনো feature বলতে পারবে?** হ্যাঁ, যেমন Street View, public transport তথ্য, ও places-এর review।

F. Technical Clarification

- স্লাইডের তথ্য সঠিক। তবে **Speaker Notes ভুল**: এই স্লাইডের Notes আসলে পুরো system-এর সারাংশ, যা এই স্লাইডের বিষয় নয়। **Notes ব্যবহার করবেন না**, উপরের Speech ব্যবহার করুন।
- "Billions of people" দিয়ে মোট ব্যবহারকারী বোঝানো হয়েছে। Google নিজে প্রকাশ্যে বলেছে Maps-এর মাসিক ব্যবহারকারী শত কোটির বেশি, তাই দাবিটি যুক্তিসংগত।

G. Delivery Tips

- **গুরুত্ব:** "পৃথিবী কখনো স্থির নয়" বাক্যটি জোর দিয়ে বলুন, এটাই সমস্যার মূল।
- **ইঙ্গিত:** তিনটি কলামে (Problem, Functional requirements, Users) পর্যায়ক্রমে হাত দিয়ে দেখান।
- **থামুন:** ছয়টি requirement পড়ার সময় প্রতিটির পর সামান্য বিরতি দিন।
- **শেষ:** "কতটা ভালোভাবে করবে" প্রশ্নটি রেখে পরের স্লাইডে যান, এতে স্বাভাবিক সংযোগ তৈরি হয়।

স্লাইড ৪: Non-Functional Requirements

A. Slide Identification

- **Slide Number:** ৪
- **Slide Title:** Non-Functional Requirements (Scale, latency, availability, durability and security)
- **উদ্দেশ্য:** system কাজ করার পাশাপাশি কতটা ভালোভাবে কাজ করবে, তার পাঁচটি মান-লক্ষ্য ঠিক করা।

B. ব্যাখ্যা

পাঁচটি স্তম্ভ আছে:

- **Scale (Billions):** কোটি কোটি ব্যবহারকারী, peak সময়ে প্রতি সেকেন্ডে লক্ষ লক্ষ request। *Peak* মানে সবচেয়ে ব্যস্ত সময়, যেমন অফিস ছুটির সময়।
- **Latency (<100 ms):** request পাঠানো থেকে উত্তর আসার সময়। স্লাইড অনুযায়ী map tile p95-এ ১০০ ms-এর কম, আর route প্রায় ১ সেকেন্ডে। **p95** মানে ১০০টির মধ্যে ৯৫টি request এই সময়ের মধ্যে শেষ হবে। গড় সময়ের চেয়ে এটি বেশি বাস্তব, কারণ ধীর request-গুলো ব্যবহারকারী অনুভব করে।
- **Availability (99.99%):** system কত সময় চালু থাকবে। ০.০১% বন্ধ থাকা মানে বছরে প্রায় ৫২ মিনিট (৩৬৫ দিন × ২৪ ঘণ্টা × ৬০ মিনিট × ০.০১% ≈ ৫২.৬ মিনিট)।
- **Durability (No loss):** একবার data সংরক্ষণ হলে তা হারাবে না। সমাধান: **multi-region replication**, অর্থাৎ একই data একাধিক দেশ বা অঞ্চলের server-এ কপি রাখা। একটি অঞ্চলে বিপর্যয় হলেও অন্য কপি থেকে যায়।

- **Security (Encrypted):** traffic encrypted, **OAuth** দিয়ে sign-in, এবং ব্যবহারকারীর privacy নিয়ন্ত্রণ।

বাস্তব উদাহরণ: বাসে বসে আপনি যখন Maps খুলে অপেক্ষা করেন, ৫ সেকেন্ডের বেশি লাগলে বিরক্ত হন। এটাই latency। আর আপনার পাঠানো live location অন্য কেউ দেখে ফেলতে পারলে সেটা security সমস্যা।

C. Presentation Speech (প্রায় ১ মিনিট)

> আগের স্লাইডে আমরা দেখেছি system কী কী কাজ করবে। এখন প্রশ্ন, সেগুলো কতটা ভালোভাবে করবে? এটাকে বলে Non-Functional Requirements, আর এখানে পাঁচটি বিষয় আছে।
>
> প্রথমটি Scale। কোটি কোটি মানুষ ব্যবহার করে, আর ব্যস্ত সময়ে প্রতি সেকেন্ডে লক্ষ লক্ষ request আসে। দ্বিতীয়টি Latency। Map tile আমাদের লক্ষ্য p95-এ ১০০ মিলিসেকেন্ডের কম। p95 মানে প্রতি একশোটি request-এর ৯৫টি এই সময়ের মধ্যে শেষ হবে। আর route বের করতে লাগবে প্রায় এক সেকেন্ড।
>
> তৃতীয়টি Availability, ৯৯.৯৯ শতাংশ। শুনতে ছোট মনে হলেও এর মানে বছরে মাত্র প্রায় ৫২ মিনিট বন্ধ থাকা। চতুর্থটি Durability, মানে data হারানো যাবে না, তাই multi-region replication ব্যবহার করে একই data একাধিক অঞ্চলে কপি রাখা হয়। আর পঞ্চমটি Security, সব traffic encrypted, OAuth দিয়ে sign-in এবং ব্যবহারকারীর privacy নিয়ন্ত্রণ।
>
> এই পাঁচটি লক্ষ্যই পরের সব architecture সিদ্ধান্তের কারণ ব্যাখ্যা করে। এবার দেখি, এত বড় পরিমাণ geographic data কীভাবে দ্রুত খোঁজা হয়।

D. Additional Knowledge

OAuth 2.0 (সংক্ষেপে): (১) *এটি কী?* এমন একটি নিয়ম যাতে আপনার পাসওয়ার্ড অন্য অ্যাপকে না দিয়েও তাকে সীমিত অনুমতি (authorization) দেওয়া যায়। (২) *কেন?* পাসওয়ার্ড শেয়ার করা বিপজ্জনক। (৩) *কীভাবে?* আপনি Google-এ লগইন করেন, Google একটি **access token** দেয়, অ্যাপ সেই token দেখিয়ে কাজ করে। (৪) *Maps-এ:* "Sign in with Google" ও API অ্যাক্সেসে। (৫) *উদাহরণ:* হোটেলের রিসেপশন আপনাকে ঘরের একটি **চাবিকার্ড** দেয়, আপনার বাড়ির মূল চাবি নয়। (৬) *শিক্ষক জিজ্ঞেস করলে:* OAuth authentication নয়, মূলত authorization-এর জন্য। Authentication বোঝাতে এর ওপরে OpenID Connect ব্যবহার হয়।

Latency বনাম Throughput: Latency একটি request-এর সময়, Throughput প্রতি সেকেন্ডে কতগুলো request সামলানো যায়। পাইপের উদাহরণ: পানি পৌঁছাতে কত সময় লাগে (latency) আর প্রতি মিনিটে কত পানি যায় (throughput)।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **৯৯.৯৯% availability কীভাবে অর্জন হয়?** Redundancy (একাধিক server), multi-region deployment, load balancer, failover এবং health check দিয়ে।
2. **p95 কেন, গড় নয়?** গড়ে ধীর request লুকিয়ে যায়। p95/p99 বাস্তব ব্যবহারকারীর অভিজ্ঞতা বেশি ভালো দেখায়।
3. **সব লক্ষ্য একসঙ্গে পাওয়া যায় কি?** পুরোপুরি নয়। Consistency বনাম availability, freshness বনাম cost, এমন trade-off থাকে (স্লাইড ১৯)।
4. **Durability ও availability একই?** না। Availability মানে এখন সেবা পাওয়া যাচ্ছে কি না, Durability মানে data হারাচ্ছে কি না।

F. Technical Clarification

- স্লাইডের সংখ্যাগুলো (<100 ms, 99.99%) **Google-এর ঘোষিত SLA নয়**, এগুলো design target। মৌখিকভাবে বলুন: "আমাদের design লক্ষ্য হিসেবে ধরা হয়েছে।"
- Notes ভুল (স্লাইড ৩-এর মতো), ব্যবহার করবেন না।

G. Delivery Tips

- **গুরুত্ব:** p95 ও ৫২ মিনিটের হিসাব, শিক্ষকের নজর কাড়ে।
- **ইঙ্গিত:** পাঁচটি কলাম বাঁ থেকে ডানে।
- **থামুন:** "৫২ মিনিট" বলার পর এক সেকেন্ড।
- **শেষ:** "এই লক্ষ্যগুলোই পরের সব সিদ্ধান্তের কারণ।"

স্লাইড ৫: Geospatial Databases & Spatial Indexing

A. Slide Identification

- **Slide Number:** ৫
- **Slide Title:** Geospatial Databases & Spatial Indexing
- **উদ্দেশ্য:** বিশাল ভৌগোলিক data কীভাবে সংরক্ষণ হয় এবং কীভাবে দ্রুত "কাছাকাছি কী আছে" খোঁজা হয়।

B. ব্যাখ্যা

মূল সমস্যা: ধরুন ১০ কোটি রেস্তোরাঁর তালিকা আছে এবং আপনি "আমার কাছের রেস্তোরাঁ" খুঁজছেন। প্রতিটির দূরত্ব হিসাব করা (**full scan**) খুব ধীর। সমাধান: **Spatial Index**, যা স্থানকে এমনভাবে সাজায় যাতে শুধু আশপাশের কয়েকটি রেকর্ড দেখলেই চলে। বইয়ের শেষের সূচিপত্র যেমন পুরো বই না ঘেঁটে পাতা খুঁজে

দেয়।

স্লাইডের ছয়টি technology:

- **PostgreSQL + PostGIS:** geographic data সংরক্ষণ ও query।
- **R-tree / GiST:** কাছাকাছি object দ্রুত খোঁজা।
- **QuadTree:** এলাকাকে বারবার চার ভাগে ভাগ করা।
- **Geohash:** coordinate-কে grid cell-এর কোডে রূপান্তর।
- **H3:** hexagonal (ষড়ভুজ) cell, aggregation-এর জন্য।
- **Redis:** কাছাকাছি ফলাফল cache করে দ্রুত উত্তর।

গুরুত্বপূর্ণ বার্তা (Notes অনুযায়ী): এগুলো একে অপরের বিকল্প নয়, একসঙ্গে কাজ করতে পারে।

C. Presentation Speech (প্রায় ২ মিনিট)

> এবার আমরা দেখব Google Maps-এর মতো system-এ geographic data কীভাবে সংরক্ষণ করা হয় এবং কীভাবে দ্রুত খোঁজা হয়। ধরুন, আপনার কাছের একটি রেস্তোরাঁ খুঁজছেন, আর data-তে আছে কোটি কোটি রেস্তোরাঁ। যদি প্রতিটির দূরত্ব একটি একটি করে মাপতে হয়, তাহলে উত্তর পেতে অনেক সময় লাগবে। এই সমস্যার সমাধানই হলো spatial indexing।
>
> একটি বইয়ের সূচিপত্র যেমন পুরো বই না ঘেঁটে সরাসরি সঠিক পাতায় নিয়ে যায়, spatial index-ও তেমনি আমাদের শুধু আশপাশের কয়েকটি রেকর্ড দেখতে দেয়।
>
> এখানে আমরা কয়েকটি technology দেখছি। PostgreSQL-এর সঙ্গে PostGIS হলো একটি extension, যা database-কে বিন্দু, রাস্তা, সীমানার মতো geographic object বুঝতে শেখায়। R-tree, যা PostGIS-এ GiST index হিসেবে থাকে, কাছাকাছি object খোঁজে বাক্সের মধ্যে বাক্স সাজিয়ে। QuadTree এলাকাকে বারবার চার ভাগে ভাগ করে। Geohash coordinate-কে একটি ছোট কোডে পরিণত করে, যেখানে কাছাকাছি জায়গার কোড প্রায় একই রকম হয়। H3 পৃথিবীকে ষড়ভুজ cell-এ ভাগ করে, যা traffic বা চাহিদার মতো aggregation-এর জন্য ভালো। আর Redis বারবার চাওয়া ফলাফল মেমোরিতে রেখে database-এর চাপ কমায়।
>
> একটা বিষয় মনে রাখা দরকার, এগুলো একে অপরের বিকল্প নয়। বাস্তবে একটি system একাধিক technology একসঙ্গে ব্যবহার করতে পারে। মূল কথা, spatial indexing আমাদের যত বেশি data-ই থাকুক, খুব কম record পরীক্ষা করে উত্তর খুঁজে দেয়। এবার দেখি, এই data ব্যবহার করে route কীভাবে বের হয়।

D. Additional Knowledge (বিস্তারিত)

১. PostGIS — (১) *কী?* PostgreSQL database-এর geospatial extension। (২) *কেন?* সাধারণ database "দূরত্ব" বা "সীমানার ভেতরে" বোঝে না। (৩) *কীভাবে?* `geometry` / `geography` টাইপ ও `ST_DWithin`, `ST_Distance` এর মতো ফাংশন দেয়। (৪) *Maps-এ:* বাস্তব নিশ্চিত নয়, তবে এমন কাজে এ ধরনের database ব্যবহার করা হয়। (৫) *উদাহরণ:* "আমার ৫০০ মিটারের মধ্যে ফার্মেসি"। (৬) *প্রশ্ন:* "Google কি PostGIS ব্যবহার করে?" উত্তর: নিশ্চিত নয়, এটি reference উদাহরণ।

২. R-tree / GiST — (১) *কী?* বাক্স (bounding box) দিয়ে সাজানো গাছ-কাঠামো। (২) *কেন?* এলাকা বা অনিয়মিত আকারের object-ও সূচিবদ্ধ করতে। (৩) *কীভাবে?* কাছাকাছি object-কে একটি বড় বাক্সে গুচ্ছ করা হয়, বাক্সগুলোকে আবার বড় বাক্সে। খুঁজতে গেলে যে বাক্সের সঙ্গে query এলাকা মেলে না, তা বাদ। (৪) *Maps-এ:* রাস্তা, বিল্ডিং, সীমানা খুঁজতে। (৫) *উদাহরণ:* বড় ফাইল ক্যাবিনেটে ড্রয়ার → ফোল্ডার → কাগজ। (৬) *প্রশ্ন:* **GiST** মানে Generalized Search Tree, PostgreSQL-এর একটি সাধারণ index কাঠামো যার ওপর R-tree তৈরি।

৩. QuadTree — (১) এলাকাকে চার ভাগে (উত্তর-পূর্ব, উত্তর-পশ্চিম ইত্যাদি) ভাগ করা গাছ। (২) যে ঘর বেশি ঘনবসতি, তাকে আরও ভাগ করা হয়। (৩) ঢাকার মতো ঘন এলাকা গভীর ভাগে, সুন্দরবন কম ভাগে থাকে। (৪) in-memory spatial search-এ ভালো। (৫) *প্রশ্ন:* QuadTree vs R-tree? QuadTree স্থানকে ভাগ করে (space partitioning), R-tree object-কে গুচ্ছ করে (data partitioning)।

৪. Geohash — (১) অক্ষাংশ-দ্রাঘিমাংশকে base32 স্ট্রিং-এ রূপান্তর। (২) স্ট্রিং যত লম্বা, এলাকা তত ছোট। (৩) কাছাকাছি জায়গার prefix প্রায় একই। তাই একটি সাধারণ database index দিয়েই "কাছাকাছি" খোঁজা যায়। (৪) *সীমাবদ্ধতা:* সীমানার দুই পাশের জায়গা কাছে হয়েও prefix আলাদা হতে পারে, তাই আশপাশের cell-ও দেখতে হয়। (৫) *প্রশ্ন:* Geohash বনাম H3? Geohash চতুর্ভুজ, H3 ষড়ভুজ। ষড়ভুজের সব প্রতিবেশী সমদূরত্বে, তাই aggregation ও smoothing ভালো হয়।

৫. H3 — Uber-এর তৈরি hexagonal hierarchical grid। Surge pricing, traffic heatmap-এর মতো কাজে ব্যবহৃত। এটি Google-এর নিজস্ব নয়।

৬. Redis — (১) *কী?* in-memory key-value store, অর্থাৎ ডেটা RAM-এ থাকে। (২) *কেন?* ডিস্কের চেয়ে অনেক দ্রুত। (৩) *কীভাবে?* key দিয়ে value রাখে। এতে `GEOADD` / `GEOSEARCH` কমান্ডও আছে। (৪) *Maps-এ:* জনপ্রিয় এলাকার ফলাফল cache করা। (৫) *উদাহরণ:* বাবুর্চি জনপ্রিয় খাবার আগে থেকে রান্না করে রাখে। (৬) *প্রশ্ন:* Cache পুরোনো হলে? TTL ও invalidation দিয়ে নতুন করা হয় (স্লাইড ১৭)।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **Spatial index কেন দরকার?** Full scan এড়াতে, যাতে কয়েকটি record দেখেই উত্তর মেলে।
2. **সব technology একসঙ্গে কেন?** প্রতিটির কাজ আলাদা: সংরক্ষণ (PostGIS), খোঁজা (R-tree), grid (Geohash/H3), দ্রুত উত্তর (Redis)।
3. **Bottleneck কোথায়?** ঘন এলাকা (hot spot), এবং বিশাল data-র জন্য একটি database যথেষ্ট নয়, তাই sharding লাগে।

4. **Scale কীভাবে?** Data-কে অঞ্চল (geohash prefix/cell) অনুযায়ী shard করে একাধিক server-এ ভাগ করা।
5. **Google কি এগুলোই ব্যবহার করে?** নিশ্চিত নই। এগুলো পরিচিত উদাহরণ।

F. Technical Clarification

- স্লাইডে এগুলো **উদাহরণ technology**। Google Maps-এর নিশ্চিত stack নয়।
- **H3 Uber-এর**, Google-এর নয়। এটি আলাদাভাবে উল্লেখ করুন।
- স্লাইড ২-এর transition ঠিক আছে, কিন্তু এই স্লাইডের পরের transition-এ বলবেন "data কীভাবে খোঁজা হয় দেখলাম, এবার route দেখি।"

G. Delivery Tips

- **গুরুত্ব:** সূচিপত্রের উপমা ও "এগুলো বিকল্প নয়, একসঙ্গে কাজ করে।"
- **ইঙ্গিত:** ছয়টি logo বাঁ থেকে ডানে ধরে দেখান।
- **থামুন:** সমস্যাটা (কোটি রেস্তোরাঁ) বলার পর, যাতে সবাই সমস্যাটা অনুভব করে।
- **শেষ:** "উত্তরটা অল্প record দেখেই বের হয়।"

স্লাইড ৬: Routing & Navigation

A. Slide Identification

- **Slide Number:** ৬
- **Slide Title:** Routing & Navigation (Finding the best route, in real-time)
- **উদ্দেশ্য:** ব্যবহারকারী গন্তব্য দিলে কীভাবে সেরা রাস্তা বের হয় তা দেখানো।

B. ব্যাখ্যা

স্লাইডে চার ধাপের প্রক্রিয়া আছে (**Input → Processing → Output** আকারে):

1. **User Request (Input):** ব্যবহারকারী গন্তব্য খোঁজে বা directions চায়।
2. **Routing Service (Processing):** রাস্তার জাল (road network) বিশ্লেষণ করে algorithm দিয়ে সম্ভাব্য পথ বের করে।
3. **Real-Time Data (Processing):** যানজট, দুর্ঘটনা, রাস্তা বন্ধ বিবেচনা করে প্রতিটি রাস্তার "খরচ" (cost) সমন্বয় করে।
4. **Optimized Route (Output):** দ্রুততম ও নিরাপদ পথ, ETA, দূরত্ব ও turn-by-turn নির্দেশনা।

Graph মডেল: মানচিত্রকে **graph** ভাবা হয়। চৌরাস্তা/মোড় = **node (vertex)**, দুই মোড়ের মধ্যে রাস্তা = **edge**। প্রতিটি edge-এর একটি **weight/cost** থাকে (দূরত্ব বা সময়)। একমুখী রাস্তা = **directed edge**। কাজ: A node থেকে B node-এ সর্বনিম্ন মোট cost-এর পথ খোঁজা।

মনে রাখুন: সবচেয়ে ছোট পথ সবসময় সবচেয়ে দ্রুত নয়। ১০ কিমি ফাঁকা রাস্তা ৫ কিমি জ্যামের রাস্তার চেয়ে দ্রুত হতে পারে। তাই cost হিসেবে "সময়" নেওয়া হয়, আর সময় real-time traffic দিয়ে ঠিক হয়।

C. Presentation Speech (প্রায় ২ মিনিট)

> আগের স্লাইডে আমরা দেখেছি data কীভাবে সংরক্ষণ ও খোঁজা হয়। এবার দেখব সেই data দিয়ে Google Maps কীভাবে আমাদের পথ দেখায়।
>
> যখন আমরা গন্তব্য দিই, request চলে যায় Routing Service-এ। এই service পুরো রাস্তার নেটওয়ার্ককে একটি graph হিসেবে দেখে। এখানে প্রতিটি মোড় বা চৌরাস্তা একটি node, আর দুই মোড়ের মধ্যে রাস্তা একটি edge। প্রতিটি edge-এর একটি cost আছে, যেমন সেই রাস্তা পার হতে কত সময় লাগবে।
>
> সেরা পথ বের করতে এখানে Dijkstra's ও A* এর মতো algorithm ব্যবহার করা হয়। Dijkstra শুরুর বিন্দু থেকে চারদিকে ধাপে ধাপে ছড়িয়ে সবচেয়ে কম খরচের পথ বের করে। A* একই কাজ করে, তবে গন্তব্যের দিকে একটি অনুমান বা heuristic ব্যবহার করে, তাই অপ্রয়োজনীয় রাস্তা কম দেখতে হয়।
>
> কিন্তু সবচেয়ে ছোট পথ সবসময় সবচেয়ে দ্রুত নয়। তাই দ্বিতীয় ধাপে real-time data আসে। যানজট, দুর্ঘটনা বা রাস্তা বন্ধের তথ্য দিয়ে প্রতিটি রাস্তার cost বদলানো হয়। শেষে আমরা পাই optimized route, সঙ্গে ETA, দূরত্ব আর turn-by-turn নির্দেশনা।
>
> মূল কথা, আধুনিক navigation হলো তিনটি জিনিসের মিলন: geographic data, graph algorithm এবং real-time traffic। এবার দেখি, এই রাস্তাগুলো পর্দায় কীভাবে দ্রুত আঁকা হয়।

D. Additional Knowledge (বিস্তারিত)

Dijkstra's Algorithm

1. **কী?** Edge-এর cost অঋণাত্মক হলে এক node থেকে অন্য সব node-এর সর্বনিম্ন cost-এর পথ বের করার algorithm।
2. **কেন?** সঠিক ও নির্ভরযোগ্য, সর্বনিম্ন cost-এর পথ নিশ্চিত করে।
3. **কীভাবে?** (ক) শুরুর node-এর দূরত্ব ০, বাকি সব অসীম ধরা হয়। (খ) অদেখা node-দের মধ্যে যার দূরত্ব সবচেয়ে কম, সেটি বেছে নেওয়া হয়। (গ) তার প্রতিবেশীদের দূরত্ব হালনাগাদ করা হয় (যদি নতুন পথ ছোট হয়)। (ঘ) গন্তব্য বাছাই হলে থামা।
4. **Maps-এ:** road graph-এ মূল ভিত্তি।

5. **সহজ উদাহরণ:** A→B ৪ মিনিট, A→C ২ মিনিট, C→B ১ মিনিট। A থেকে শুরু। C (২) প্রথমে বাছা হয়। তারপর C→B দিয়ে B-র দূরত্ব ২+১=৩, যা সরাসরি ৪-এর চেয়ে ভালো। তাই B-র দূরত্ব ৩।
6. **শিক্ষকের প্রশ্ন:** *সীমাবদ্ধতা কী?* ঋণাত্মক weight সমর্থন করে না, আর বিশ্বজোড়া graph-এ সরাসরি চালালে খুব ধীর, কারণ চারদিকেই ছড়ায়।

A* (A-star) Algorithm

1. **কী?** Dijkstra-র উন্নত রূপ, যা গন্তব্যের দিকে "দিকনির্দেশক অনুমান" (heuristic) ব্যবহার করে।
2. **কেন?** Dijkstra চারদিকে সমানভাবে ছড়ায়, A* গন্তব্যমুখী হয়ে কম node দেখে।
3. **কীভাবে?** প্রতিটি node-এর জন্য `f = g + h`। `g` = শুরু থেকে এ পর্যন্ত আসা cost, `h` = এখান থেকে গন্তব্যের আনুমানিক cost (যেমন সরলরেখার দূরত্ব)। যার `f` সবচেয়ে কম, তাকে আগে দেখা হয়।
4. **Maps-এ:** দ্রুত route বের করতে। (স্লাইড ২২-এর Reference [4])
5. **উদাহরণ:** ঢাকা থেকে চট্টগ্রাম যেতে চাইলে সিলেটের দিকের রাস্তা আর দেখার দরকার নেই।
6. **প্রশ্ন:** Heuristic কখন সঠিক? সেটি **admissible** হতে হবে, অর্থাৎ বাস্তব cost-এর চেয়ে বেশি অনুমান করবে না।

Contraction Hierarchies (Additional Knowledge, স্লাইডে নেই) স্লাইড ২২-এর Reference [3]। এটি আগে থেকে (preprocessing) "গুরুত্বহীন" node-গুলোকে shortcut edge দিয়ে সংকুচিত করে রাখে, তাই query-র সময় কয়েক মিলিসেকেন্ডে বিশাল road network-এও উত্তর মেলে। উদাহরণ: গ্রামের গলি থেকে আগে হাইওয়েতে ওঠা, তারপর শহর থেকে আবার গলিতে নামা। এই উঁচু-নিচু ধরনটাই hierarchy। বাস্তবে বিশ্বজোড়া নেটওয়ার্কে Dijkstra/A* সরাসরি না চালিয়ে এ ধরনের প্রযুক্তি লাগে, এটি বললে শিক্ষক খুশি হবেন।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. *Dijkstra আর A এর পার্থক্য?** Dijkstra চারদিকে ছড়ায়, A* heuristic দিয়ে গন্তব্যমুখী হয়।
2. **Graph-এ কী কী আছে?** Node = মোড়/স্থান, Edge = রাস্তা, Weight = সময় বা দূরত্ব।
3. **Traffic কীভাবে route-কে প্রভাবিত করে?** Edge-এর cost সময়ের সঙ্গে বদলায়।
4. **Bottleneck?** খুব বড় road graph-এ route search ধীর হয়ে যাওয়া (স্লাইড ১৯)।
5. **Scale কীভাবে?** Graph-কে অঞ্চলে ভাগ করা, preprocessing (Contraction Hierarchies), এবং আগের route cache করা।

F. Technical Clarification

- স্লাইডের "Directions API" একটি public API-এর নাম। এটি Google-এর সত্যিকার API, কিন্তু ভেতরের algorithm স্লাইডের উদাহরণ মাত্র।
- "Dijkstra, A* for efficient routing" ধারণাগতভাবে ঠিক, তবে বিশ্বস্তরের সিস্টেমে শুধু এগুলোই যথেষ্ট নয়, উন্নত পদ্ধতি লাগে। একথা মুখে বলে নিন।

- Notes ঠিক আছে, তবে Notes-এ "Google Cloud" এর ভূমিকা আলাদা করে বলা নেই।

G. Delivery Tips

- **গুরুত্ব:** "সবচেয়ে ছোট ≠ সবচেয়ে দ্রুত"।
- **ইঙ্গিত:** চারটি ধাপের তীর বাঁ থেকে ডানে।
- **থামুন:** graph-এর ব্যাখ্যার (node, edge) পর।
- **শেষ:** "graph + algorithm + real-time traffic"।

স্লাইড ৭: Map Rendering & Tile Delivery

A. Slide Identification

- **Slide Number:** ৭
- **Slide Title:** Map Rendering & Tile Delivery (Fast, scalable and smooth map experience)
- **উদ্দেশ্য:** পুরো পৃথিবীর মানচিত্র ফোনে কীভাবে মসৃণভাবে দেখা যায় তা বোঝানো।

B. ব্যাখ্যা

মূল ধারণা: পুরো পৃথিবীর মানচিত্র একসঙ্গে ডাউনলোড করা অসম্ভব। তাই মানচিত্রকে **tile** (ছোট বর্গাকার টুকরো, যেমন ২৫৬×২৫৬ পিক্সেল) এ ভাগ করা হয়। ফোন শুধু যে tile পর্দায় দেখা যাচ্ছে সেগুলোই চায়।

Zoom level: প্রতিটি zoom-এ আলাদা tile-সেট থাকে। Zoom ০-তে পুরো পৃথিবী ১টি tile। প্রতি zoom বাড়ালে tile-সংখ্যা ৪ গুণ হয় (z স্তরে ৪^z)। সবচেয়ে প্রচলিত ধারণা **z/x/y** ঠিকানা, মানে zoom, কলাম, সারি।

স্লাইডের চার ধাপ:

1. **Tile-Based Approach:** বিশ্বকে tile-এ ভাগ।
2. **Efficient Delivery:** CDN দিয়ে বিশ্বজুড়ে দ্রুত পরিবেশন।
3. **Dynamic Rendering:** marker, label ও real-time তথ্য যোগ।
4. **Smooth Experience:** শুধু যতটা দরকার, ততটাই লোড।

Raster বনাম Vector Tile:

- **Raster tile:** তৈরি করা ছবি (PNG/JPEG)। সহজ, কিন্তু প্রতিটি zoom-এ আলাদা ছবি লাগে এবং ঘোরানো/স্টাইল পরিবর্তন কঠিন।
- **Vector tile:** ছবি নয়, রাস্তা-বিল্ডিংয়ের জ্যামিতিক data। ফোন নিজে আঁকে। ফলে আকারে ছোট, ঘোরানো যায়, dark mode-এর মতো স্টাইল বদলানো সহজ।

C. Presentation Speech (প্রায় ২ মিনিট)

> এখন দেখব এত বিশাল পৃথিবীর মানচিত্র আমাদের ফোনে কীভাবে মসৃণভাবে চলে। ভাবুন, যদি পুরো পৃথিবীর মানচিত্র একসঙ্গে ডাউনলোড করতে হতো, তাহলে অ্যাপ খুলতেই কয়েক ঘণ্টা লাগত। এটা সমাধান হয়েছে tile দিয়ে।
>
> Tile মানে মানচিত্রের ছোট বর্গাকার টুকরো, সাধারণত ২৫৬ বাই ২৫৬ পিক্সেল। যেমন, একটি বড় দেয়ালে অনেকগুলো ছোট টাইল্স লাগিয়ে পুরো ছবি তৈরি হয়। আমরা যখন মানচিত্র সরাই বা zoom করি, অ্যাপ শুধু পর্দায় যে tile দেখা যাচ্ছে সেগুলো চেয়ে নেয়।
>
> এই tile-গুলো আগে থেকে তৈরি করে CDN বা Content Delivery Network-এর মাধ্যমে বিশ্বজুড়ে ছড়িয়ে রাখা হয়। ফলে বাংলাদেশের কোনো ব্যবহারকারী প্রায়ই কাছের server থেকেই tile পেয়ে যান, আমেরিকার মূল server পর্যন্ত যেতে হয় না। জনপ্রিয় tile আবার cache-এও থাকে, তাই আরও দ্রুত।
>
> Tile দুই ধরনের: Raster tile, যা তৈরি করা ছবি, আর Vector tile, যেখানে শুধু রাস্তা-বিল্ডিংয়ের data থাকে এবং ফোন নিজে আঁকে। Vector tile আকারে ছোট, আর স্টাইল বদলানো সহজ। অ্যাপ শেষে এই tile-গুলোর ওপর marker, label আর live traffic বসিয়ে একটি অখণ্ড মানচিত্র বানায়।
>
> মূল কথা, tile-ভিত্তিক পদ্ধতিতে অপ্রয়োজনীয় data পাঠাতে হয় না, তাই লোড দ্রুত হয় এবং system বিশ্বস্তরে scale করে। এবার দেখি, মানচিত্রে কোনো জায়গা কীভাবে খোঁজা হয়।

D. Additional Knowledge (বিস্তারিত)

CDN (Content Delivery Network)

1. **কী?** বিশ্বজুড়ে ছড়ানো edge server-এর নেটওয়ার্ক, যারা মূল server-এর কনটেন্টের কপি রাখে।
2. **কেন?** দূরত্ব কমলে latency কমে, আর মূল server-এর চাপও কমে।
3. **কীভাবে?** ব্যবহারকারীর request সবচেয়ে কাছের edge-এ যায়। সেখানে কপি থাকলে (**cache hit**) সঙ্গে সঙ্গে উত্তর। না থাকলে (**cache miss**) মূল server থেকে এনে রাখে, পরেরবার দ্রুত।
4. **Maps-এ:** tile ও ছবি পরিবেশনে।
5. **উদাহরণ:** ঢাকার দোকানে জনপ্রিয় পণ্য আগে থেকে রাখা থাকলে আর কারখানা (আমেরিকা) থেকে আনতে হয় না।
6. **প্রশ্ন:** CDN বনাম Cache? CDN ভৌগোলিকভাবে ছড়ানো cache। সাধারণ cache একটি জায়গায় থাকতে পারে।

Caching (Redis): দ্রুত উত্তরের জন্য। বিস্তারিত স্লাইড ৫ ও ১৭।

Web Mercator (Additional): পৃথিবীর গোলাকার পৃষ্ঠকে সমতল মানচিত্রে আনার একটি projection, যা প্রায় সব web map ব্যবহার করে। এতে মেরুর কাছে আকার বিকৃত হয় (গ্রিনল্যান্ড বড় দেখায়)।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **Tile কেন, পুরো ছবি নয়?** Data কম লাগে, দ্রুত লোড হয়, cache করা সহজ।
2. **Vector tile-এর সুবিধা?** আকারে ছোট, স্টাইল পরিবর্তনযোগ্য, মসৃণ zoom ও rotation।
3. **Raster tile কখন?** সহজ প্রয়োগ বা satellite ছবির মতো ছবিনির্ভর ক্ষেত্রে।
4. **Bottleneck?** ব্যস্ত শহরে জনপ্রিয় tile (hot tile) (স্লাইড ১৯)।
5. **Cache পুরোনো হলে?** Tile-এর version/TTL দিয়ে নতুন সংস্করণ আনা হয়।

F. Technical Clarification

- স্লাইডে **"CDN (Cloudflare)"** ও **"Mapbox / Google Maps Tiles"** আছে। Google Maps নিজের tile বা CDN-এ Cloudflare/Mapbox ব্যবহার করবে, এটি বিশ্বাসযোগ্য নয়। এগুলোকে "এ ধরনের system-এ ব্যবহৃত পরিচিত উদাহরণ" বলুন। স্লাইড ১৭ আবার "Google Cloud CDN" বলেছে, তাই অসঙ্গতি আছে।
- Notes সঠিক।

G. Delivery Tips

- **গুরুত্ব:** দেয়ালের টাইল্সের উপমা ও "শুধু যা দরকার তা-ই লোড"।
- **ইঙ্গিত:** চার ধাপের timeline ও tile-এর ছবির দিকে।
- **থামুন:** raster vs vector বলার আগে একটু থামুন।
- **শেষ:** "কম data, দ্রুত লোড, সহজ scale।"

স্লাইড ৮: Places Search & Discovery

A. Slide Identification

- **Slide Number:** ৮
- **Slide Title:** Places Search & Discovery (Find the right place, instantly)
- **উদ্দেশ্য:** ব্যবহারকারী কোনো জায়গা খুঁজলে কীভাবে সঠিক, কাছের ও উপযোগী ফলাফল দেওয়া হয় তা দেখানো।

B. ব্যাখ্যা

স্লাইডে চার ধাপের pipeline (**Input → Processing → Output**):

1. **Data Source:** জায়গা, ব্যবসা ও POI (Point of Interest, অর্থাৎ আগ্রহের স্থান, যেমন হাসপাতাল, দোকান)। এগুলো database-এ সংরক্ষিত।

2. **Search Engine: Full-text + Geospatial Search**। Full-text মানে লেখার মিল ("pizza" লিখলে pizza-র দোকান), Geospatial মানে অবস্থানের মিল (আপনার কাছের)। দুটো একসঙ্গে চলে।
3. **Ranking:** relevance (কতটা মেলে), distance (কত কাছে), quality (রেটিং) অনুযায়ী ফলাফল সাজানো।
4. **Results:** ব্যক্তিগতকৃত ও স্থানীয় ফলাফল।

বাড়তি features: **Nearby Search** (আশপাশে খোঁজা), **Top Results** (রেটিং ও জনপ্রিয়তা), **Real-Time Updates** (নতুন জায়গা, পরিবর্তন), **Smart Filters** (এখন খোলা, দাম, ধরন)।

উদাহরণ: "coffee near me" লিখলে (ক) কীওয়ার্ড "coffee" দিয়ে দোকান ছাঁকা হয়, (খ) আপনার অবস্থানের আশপাশের দোকান রাখা হয়, (গ) রেটিং, দূরত্ব, খোলা আছে কি না দেখে সাজানো হয়, (ঘ) আপনার আগের পছন্দ বিবেচনা করা হতে পারে।

C. Presentation Speech (প্রায় ১.৫ মিনিট)

> আমরা সবাই Maps-এ নিয়মিত কোনো জায়গা খুঁজি। যেমন "কাছের ফার্মেসি" বা "ভালো রেস্তোরাঁ"। এই সাধারণ কাজটার পেছনেও একটি পুরো pipeline আছে।
>
> প্রথমে আছে data source, যেখানে জায়গা, ব্যবসা এবং POI সংরক্ষিত থাকে। POI মানে Point of Interest, অর্থাৎ হাসপাতাল, দোকান বা ক্যাফের মতো আগ্রহের জায়গা। যখন আমরা কিছু লিখি, search engine কাজ শুরু করে। এখানে দুই ধরনের খোঁজ একসঙ্গে চলে: লেখার মিল, যাকে বলে full-text search, আর অবস্থানের মিল, যাকে বলে geospatial search।
>
> কিন্তু শুধু খুঁজে পেলেই হবে না, সাজানোও দরকার। এই কাজ করে ranking, যা প্রাসঙ্গিকতা, দূরত্ব আর মান দেখে ফলাফল সাজায়। সবশেষে আমরা পাই ব্যক্তিগতকৃত ও স্থানীয় ফলাফল।
>
> এর সঙ্গে আছে কিছু সুবিধা, যেমন nearby search, রেটিং ও জনপ্রিয়তা দেখে সেরা ফলাফল, নতুন জায়গার real-time update, আর filter, যেমন এখন খোলা আছে কি না বা দাম কেমন। মূল কথা, search, location data আর ranking একসঙ্গে বলেই জায়গা খোঁজা দ্রুত ও কাজের হয়। এবার দেখি, রাস্তায় যানজট কীভাবে ধরা পড়ে।

D. Additional Knowledge

Full-text Search ও Inverted Index:

1. *কী?* লেখার মধ্যে শব্দ খোঁজার পদ্ধতি। 2. *কেন?* প্রতিটি নথি পড়ে খোঁজা ধীর। 3. *কীভাবে?* **Inverted index** বইয়ের পিছনের শব্দসূচির মতো: প্রতিটি শব্দের পাশে লেখা থাকে সেই শব্দ কোন কোন জায়গার বর্ণনায় আছে। 4. *Maps-এ:* জায়গার নাম, ঠিকানা, ক্যাটাগরি খোঁজা। 5. *উদাহরণ:* "pizza" শব্দ → [দোকান ১, ৫, ৯]। 6. *প্রশ্ন:* সাধারণ উদাহরণ Elasticsearch/Lucene, তবে Google-এর নিজস্ব নিশ্চিত নয়।

Ranking: সাধারণত একাধিক সংকেতের সমন্বয়, যেমন text relevance, দূরত্ব, রেটিং, জনপ্রিয়তা, এখন খোলা কি না। কিছু system এটি **Learning to Rank** (ML) দিয়ে করে। **Personalization** মানে আপনার পুরোনো অনুসন্ধানের ভিত্তিতে ফলাফল মানিয়ে নেওয়া, তবে এর জন্য গোপনীয়তা নিয়ন্ত্রণ দরকার (স্লাইড ১৮)।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **Full-text ও geospatial search কেন দুটোই?** একা কোনোটি যথেষ্ট নয়: "কী" (text) ও "কোথায়" (location) দুটোই জানতে হয়।
2. **Ranking কীভাবে হয়?** Relevance, distance, quality ও personalization সংকেতের সমন্বয়ে।
3. **নতুন জায়গা কীভাবে যোগ হয়?** ব্যবসার মালিক, ব্যবহারকারী ও data pipeline থেকে (স্লাইড ২০)।
4. **Bottleneck?** জনবহুল এলাকায় একসঙ্গে অনেক query এবং index তাজা রাখা।
5. **ভুয়া বা পুরোনো তথ্য?** যাচাই ও ব্যবহারকারীর রিপোর্ট দিয়ে।

F. Technical Clarification

- স্লাইডটি ধারণাগত। কোন search engine ব্যবহৃত হয়, স্লাইডে বলা নেই, তাই সুনির্দিষ্ট নাম দাবি করবেন না।
- Notes সঠিক।

G. Delivery Tips

- **গুরুত্ব:** full-text + geospatial একসঙ্গে, আর ranking-এর তিন সংকেত।
- **ইঙ্গিত:** চার ধাপের তীর ধরে বাঁ থেকে ডানে।
- **থামুন:** POI-র ব্যাখ্যার পর।
- **শেষ:** "search + location + ranking = উপযোগী ফলাফল।"

স্লাইড ৯: Real-Time Traffic & Live Updates

A. Slide Identification

- **Slide Number:** ৯
- **Slide Title:** Real-Time Traffic & Live Updates (Stay informed, avoid delays)
- **উদ্দেশ্য:** যানজটের তথ্য কীভাবে সংগৃহীত হয়, প্রক্রিয়া হয় এবং পথ আপডেট করে।

B. ব্যাখ্যা

প্রবাহ (চার ধাপ):

1. **Live Data:** sensors, user reports, **crowdsourcing** (অনেক মানুষের ডিভাইস থেকে তথ্য সংগ্রহ)।
2. **Traffic Analytics:** তথ্য প্রক্রিয়া ও ভবিষ্যৎ আন্দাজ।
3. **Map Updates:** রিয়েল-টাইমে মানচিত্রে লাল/হলুদ/সবুজ দেখানো।
4. **Better Routes:** Smart Rerouting ও ETA update।

বৈশিষ্ট্য: Live Traffic, Incident Alerts (দুর্ঘটনা/রাস্তা বন্ধ), Smart Rerouting, ETA Updates।

সহজ ধারণা: একটি রাস্তায় যদি ৫০টি ফোন প্রায় ৫ কিমি/ঘণ্টা গতিতে চলে, বোঝা যায় সেখানে জ্যাম। অথচ কেউ কিছু রিপোর্ট করেনি। এভাবে ডিভাইসের গতি থেকেই যানজট ধরা পড়ে।

C. Presentation Speech (প্রায় ১.৫ মিনিট)

> Google Maps-এর সবচেয়ে উপকারী সুবিধাগুলোর একটি হলো real-time traffic। আমরা পথে বের হওয়ার আগেই জানতে পারি কোন রাস্তায় জ্যাম আর কোনটি ফাঁকা।
>
> এর শুরুটা live data দিয়ে। এখানে তথ্য আসে sensor, ব্যবহারকারীর রিপোর্ট এবং crowdsourcing থেকে। Crowdsourcing মানে অনেক ব্যবহারকারীর ফোন থেকে আসা চলাচলের তথ্য। যেমন কোনো রাস্তায় সবার গতি হঠাৎ কমে গেলে বোঝা যায়, সেখানে সমস্যা আছে। এরপর traffic analytics ধাপে এই তথ্য প্রক্রিয়া করে যানজট ও সম্ভাব্য পরিস্থিতি আন্দাজ করা হয়।
>
> তারপর map update ধাপে মানচিত্রে যানজটের রং বদলে যায়। শেষ ধাপে এই তথ্য ব্যবহার করে আরও ভালো পথ খোঁজা হয়। তাই যদি সামনে দুর্ঘটনা বা রাস্তা বন্ধ থাকে, system নতুন পথ দেখায় এবং পৌঁছানোর সময় আপডেট করে।
>
> এখানে চারটি জিনিস ব্যবহারকারী সরাসরি পায়: live traffic, incident alert, smart rerouting এবং ETA update। মূল কথা, real-time data থাকায় navigation দ্রুত ও নির্ভরযোগ্য হয়। এবার দেখি, ETA কীভাবে এত নির্ভুলভাবে অনুমান করা হয়।

D. Additional Knowledge

Stream Processing (Additional): Live data ধারাবাহিকভাবে আসে, তাই এটি ব্যাচে নয়, প্রবাহে প্রক্রিয়া করা হয়। এ ধরনের কাজে সাধারণ উদাহরণ Pub/Sub, Kafka, Dataflow/Beam (স্লাইড ১৬, ২০)। উপমা: নদী থেকে প্রতি মুহূর্তে পানি পরীক্ষা করা, বালতিতে জমিয়ে পরে পরীক্ষা করা নয়।

গোপনীয়তা: crowdsourcing-এ ব্যক্তির অবস্থান সরাসরি প্রকাশ করা হয় না। সাধারণত তথ্য সমষ্টিগত ও বেনামী করে ব্যবহার করা হয়। এটি বললে নিরাপত্তা-সচেতনতা দেখা যায়।

Smart Rerouting কীভাবে? নতুন traffic cost দিয়ে route আবার হিসাব করা হয়। যে পথ আগের চেয়ে নির্ধারিত সময় বেশি ভালো হলে প্রস্তাব দেওয়া হয়।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **Traffic data কোথা থেকে আসে?** Sensor, ব্যবহারকারীর ডিভাইস (crowdsourcing), রিপোর্ট ও সহযোগী উৎস।
2. **ব্যবহারকারীর privacy কীভাবে বজায় থাকে?** তথ্য সমষ্টিগত/বেনামী করা হয়, এবং ব্যবহারকারী নিয়ন্ত্রণ করতে পারে (স্লাইড ১৮)।
3. **Bottleneck?** কোটি কোটি live update একসঙ্গে প্রক্রিয়া করা।
4. **কম ব্যবহারকারীর এলাকায়?** তথ্য কম, তাই ঐতিহাসিক pattern-এ বেশি নির্ভর করতে হয় (স্লাইড ১০)।

F. Technical Clarification

- স্লাইডটি ধারণাগতভাবে ঠিক। "Sensors" সব দেশে সমানভাবে নেই, বহু জায়গায় মূলত ফোনের তথ্যই উৎস।
- Notes সঠিক।

G. Delivery Tips

- **গুরুত্ব:** crowdsourcing (ফোনের গতি → জ্যাম)।
- **ইঙ্গিত:** live traffic map-এর রঙিন রেখা দেখান।
- **থামুন:** crowdsourcing ব্যাখ্যার পর।
- **শেষ:** "real-time data = নির্ভরযোগ্য navigation।"

স্লাইড ১০: ETA Prediction & Machine Learning

A. Slide Identification

- **Slide Number:** ১০
- **Slide Title:** ETA Prediction & Machine Learning
- **উদ্দেশ্য:** ML দিয়ে কীভাবে পৌঁছানোর সময় নির্ভুলভাবে অনুমান করা হয়।

B. ব্যাখ্যা

ETA = Estimated Time of Arrival। প্রক্রিয়া (**Input → Processing → Output**):

1. **Collect Data (Input):** ঐতিহাসিক traffic, live traffic, context (আবহাওয়া, ইভেন্ট, ছুটি), রাস্তার বৈশিষ্ট্য (ধরন, দৈর্ঘ্য, বাঁক, উচ্চতা)।
2. **Train ML Models:** এই data দিয়ে model-কে শেখানো।
3. **Predict Travel Time (Output):** ভ্রমণের সময়ের অনুমান।
4. **Update Routes & ETA:** পরিস্থিতি বদলালে হালনাগাদ।

ML Techniques: **Gradient Boosting/GBDT** (XGBoost, LightGBM), **Deep Learning** (LSTM, GNN), **Time Series Models**।

সহজ উদাহরণ: সোমবার সকাল ৯টায় একটি রাস্তা সাধারণত ধীর। বৃষ্টি হলে আরও ধীর। এই সব সংকেত থেকে model শেখে কত সময় লাগবে।

C. Presentation Speech (প্রায় ২ মিনিট)

> আমরা সবাই Maps-এ "২৫ মিনিট" দেখি, আর বেশিরভাগ সময় সেটা প্রায় মিলে যায়। এটা সম্ভব হয় machine learning-এর কারণে।
>
> প্রথমে data সংগ্রহ। এখানে চার ধরনের data লাগে: আগের দিনগুলোর traffic pattern, যেমন কোন দিন কোন সময়ে রাস্তা কেমন থাকে; এখনকার live traffic; আবহাওয়া, ছুটি বা অনুষ্ঠানের মতো পরিস্থিতি; আর রাস্তার ধরন, দৈর্ঘ্য, বাঁক বা উচ্চতার মতো বৈশিষ্ট্য।
>
> এই data দিয়ে ML model-কে train করা হয়। Train মানে model অতীতের উদাহরণ দেখে শেখে কোন পরিস্থিতিতে কত সময় লেগেছিল। এখানে ব্যবহৃত কয়েকটি পদ্ধতি হলো Gradient Boosting, যেমন XGBoost বা LightGBM; Deep Learning, যেমন LSTM ও Graph Neural Network বা GNN; এবং Time Series Model, যা সময়ের সঙ্গে traffic-এর পরিবর্তন অনুমান করে।
>
> Train হওয়া model তারপর ভ্রমণের সময় অনুমান করে। আর পরিস্থিতি বদলালে, যেমন জ্যাম বাড়লে, ETA হালনাগাদ হয় এবং দরকারে বিকল্প পথ প্রস্তাব করা হয়। একটি উল্লেখযোগ্য বিষয়, স্লাইড ২২-এর প্রথম রেফারেন্সটি GNN দিয়ে Google Maps-এর ETA নিয়ে একটি গবেষণাপত্র। মূল কথা, ML-এর কাজ হলো ETA নির্ভুল করা ও navigation আরও নির্ভরযোগ্য করা। এবার দেখি, ফোনের GPS data কীভাবে ঠিক রাস্তায় বসানো হয়।

D. Additional Knowledge (বিস্তারিত)

Machine Learning (মূল ধারণা): (১) কম্পিউটারকে নিয়ম না লিখে উদাহরণ দেখিয়ে শেখানো। (২) কেন? ট্রাফিকের নিয়ম এত জটিল যে হাতে লেখা সম্ভব নয়। (৩) কীভাবে? Input (সময়, আবহাওয়া) → Model → Output (সময়)। ভুল হলে model নিজেকে সংশোধন করে। (৪) Maps-এ: ETA, ভিড় অনুমান। (৫) উদাহরণ: অভিজ্ঞ রিকশাওয়ালা যেমন সময়ের আন্দাজ জানেন। (৬) প্রশ্ন: Train ও Inference-এর পার্থক্য? Train মানে শেখা, Inference মানে শেখা model দিয়ে নতুন অনুমান।

Gradient Boosting (GBDT): (১) অনেকগুলো ছোট decision tree পরপর যোগ করা, যেখানে প্রতিটি আগেরটির ভুল শোধরায়। (২) টেবিল-আকৃতির data-তে খুব ভালো ও দ্রুত। (৩) উদাহরণ: প্রথম tree ২০ মিনিট বলল, বাস্তব ২৫ মিনিট। পরের tree ৫ মিনিটের ঘাটতি পূরণ করতে শেখে। (৪) প্রশ্ন: XGBoost ও LightGBM কী? এর জনপ্রিয় লাইব্রেরি।

LSTM ও Time Series: LSTM এক ধরনের neural network যা ক্রম বা সময়ের pattern মনে রাখে (যেমন গত ৩০ মিনিটে গতি কেমন বদলেছে)। Time series model সময়ের ক্রমে data দেখে ভবিষ্যৎ আন্দাজ করে।

GNN (Graph Neural Network):

1. **কী?** যে neural network graph-এর কাঠামো বোঝে। 2. **কেন?** রাস্তা নিজেই graph। একটি রাস্তার জ্যাম পাশের রাস্তাকেও প্রভাবিত করে। সাধারণ model এই সম্পর্ক ধরতে পারে না। 3. **কীভাবে?** প্রতিটি node বা অংশ প্রতিবেশীদের কাছ থেকে তথ্য নেয় (message passing), ফলে প্রতিবেশীর পরিস্থিতি জেনে নিজের অনুমান ঠিক করে। 4. **Maps-এ:** রাস্তার অংশগুলোর মধ্যে সম্পর্ক ধরে ETA অনুমান। স্লাইড ২২-এর Reference [1]: Derrow-Pinion et al. (2021), Google/DeepMind গবেষণা। 5. **উদাহরণ:** মূল সড়কে দুর্ঘটনা হলে পাশের গলিতেও ভিড় বাড়ে, GNN এই চেইন-প্রভাব বোঝে। 6. **প্রশ্ন:** "GNN কেন?" উত্তর: রাস্তার কাঠামো ও সম্পর্ক সরাসরি কাজে লাগাতে পারে।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **ML কেন, সাধারণ গড় নয়?** Traffic জটিল ও পরিবর্তনশীল। ML একসঙ্গে অনেক সংকেত বিবেচনা করে।
2. **GBDT বনাম Deep Learning?** GBDT সহজ ও দ্রুত, Deep Learning জটিল সম্পর্ক (সময়/graph) ভালো ধরে।
3. **Model ভুল করলে?** নতুন data দিয়ে নিয়মিত retrain ও live data দিয়ে সংশোধন।
4. **Bottleneck?** বিপুল data প্রক্রিয়া ও প্রতি মুহূর্তে অনুমান (inference) দ্রুত রাখা।
5. **নতুন বা অস্বাভাবিক ঘটনায়?** (ধর্মঘট, বন্যা) ঐতিহাসিক data কম কাজ করে, live data বেশি ভরসা।

F. Technical Clarification

- স্লাইডের তালিকা (GBDT, LSTM, GNN) **সম্ভাব্য পদ্ধতি**। Google সব কটাই একসঙ্গে ব্যবহার করে, এমন দাবি করবেন না। GNN-এর ক্ষেত্রে প্রকাশিত গবেষণা আছে (Reference [1]), যা এই স্লাইডের সবচেয়ে শক্ত সমর্থন।
- Notes সঠিক।

G. Delivery Tips

- **গুরুত্ব:** চার ধরনের Data ও চার ধাপ।
- **ইঙ্গিত:** "Data Used" ও "ML Techniques" দুটি অংশে আলাদাভাবে হাত রাখুন।
- **থামুন:** GNN-এর কথা বলার সময়, এটি আপনার শক্তিশালী পয়েন্ট।
- **শেষ:** "ML = নির্ভুল ETA।"

স্লাইড ১১: GPS Location Tracking & Map Matching

A. Slide Identification

- **Slide Number:** ১১
- **Slide Title:** GPS Location Tracking & Map Matching
- **উদ্দেশ্য:** কাঁচা (raw) GPS সংকেত থেকে রাস্তার ওপর নির্ভুল অবস্থান বের করা।

B. ব্যাখ্যা

সমস্যা: GPS সংকেত স্যাটেলাইট থেকে আসে। ভবন, টানেল বা মেঘের কারণে ৫–৩০ মিটার পর্যন্ত ভুল হতে পারে (স্লাইডে লেখা)। ফলে নীল বিন্দু কখনও নদীর মাঝখানে বা পাশের রাস্তায় দেখা যায়।

চার ধাপের প্রক্রিয়া (**Input → Processing → Output**):

1. **Receive GPS Signals (Input):** latitude, longitude ও সময়ের ক্রম।
2. **Filter & Clean Data:** noise কমানো (যেমন **Kalman Filter**)।
3. **Map Matching (Snap to Road):** পরিষ্কার অবস্থানকে সবচেয়ে সম্ভাব্য রাস্তার সঙ্গে মেলানো (**HMM, Viterbi**)।
4. **Accurate Real-Time Location (Output):** মসৃণ ও নির্ভুল অবস্থান।

সঙ্গে **Sensor Fusion**: GPS, Wi-Fi, Cellular ও **IMU** (Inertial Measurement Unit, অর্থাৎ ফোনের accelerometer/gyroscope) একসঙ্গে ব্যবহার।

C. Presentation Speech (প্রায় ২ মিনিট)

> আমরা Maps-এ যে নীল বিন্দুটি দেখি, সেটা কিন্তু এত সহজে নির্ভুল হয় না। GPS সংকেত স্যাটেলাইট থেকে আসে, আর বড় ভবন, টানেল বা দুর্বল সংকেতের কারণে ৫ থেকে ৩০ মিটার পর্যন্ত ভুল হতে পারে। শহরে এই ভুলের কারণে মনে হতে পারে আমরা পাশের রাস্তায় আছি।
>
> এই সমস্যা সমাধানের প্রথম ধাপ হলো GPS signal গ্রহণ করা, যেখানে আমরা শুধু latitude আর longitude পাই। দ্বিতীয় ধাপে data পরিষ্কার করা হয়, অর্থাৎ noise কমানো হয়, যার একটি পরিচিত পদ্ধতি Kalman Filter। তৃতীয় ধাপ হলো Map Matching বা snap to road। এখানে অবস্থানটিকে মানচিত্রের সবচেয়ে সম্ভাব্য রাস্তার সঙ্গে মিলিয়ে দেওয়া হয়। এর জন্য Hidden Markov Model বা HMM এবং Viterbi algorithm ব্যবহার করা যায়। শেষ ধাপে আমরা পাই মসৃণ ও নির্ভুল অবস্থান।
>
> এর সঙ্গে আছে sensor fusion, অর্থাৎ শুধু GPS নয়, Wi-Fi, মোবাইল টাওয়ার আর ফোনের নিজস্ব গতি-সেন্সরের তথ্য একসঙ্গে মেলানো। ফলে টানেলে GPS হারালেও ফোন খানিকটা অনুমান করতে পারে আমরা কোথায়।
>
> মূল কথা, এসব প্রযুক্তি মিলিয়ে navigation মসৃণ হয়, অবস্থান বিশ্বাসযোগ্য হয়, আর turn-by-turn নির্দেশনা সঠিক সময়ে আসে। এবার দেখি, ইন্টারনেট না থাকলে কী হয়।

D. Additional Knowledge (বিস্তারিত)

Kalman Filter

1. **কী?** গোলমালপূর্ণ মাপ থেকে সবচেয়ে সম্ভাব্য প্রকৃত অবস্থা অনুমান করার গাণিতিক পদ্ধতি। 2. **কেন?** GPS-এর মাপ লাফালাফি করে। 3. **কীভাবে?** দুটি জিনিস মেলায়: (ক) আগের অবস্থান ও গতি দিয়ে করা **পূর্বানুমান**, (খ) নতুন GPS মাপ। দুটির কোনটি কতটা নির্ভরযোগ্য, সে অনুযায়ী ভারসাম্য করে। 4. **Maps-এ:** অবস্থান মসৃণ করা। 5. **উদাহরণ:** গাড়ি ঘণ্টায় ৪০ কিমি গতিতে চলছে, হঠাৎ GPS ৫০ মিটার পাশে লাফ দিল। Filter বোঝে এটা সম্ভব নয়, তাই কম গুরুত্ব দেয়। 6. **প্রশ্ন:** Filter বনাম Map matching? Filter সংখ্যা মসৃণ করে, Map matching তাকে রাস্তার সঙ্গে মেলায়।

Map Matching, HMM ও Viterbi

1. **Map Matching কী?** GPS বিন্দুগুলোকে সবচেয়ে সম্ভাব্য রাস্তায় বসানো। 2. **কেন কঠিন?** পাশাপাশি দুটি রাস্তা থাকলে শুধু একটি বিন্দু দেখে বলা যায় না কোনটিতে আছি।
2. **HMM (Hidden Markov Model):** আসল অবস্থান (কোন রাস্তা) *লুকানো*, আমরা শুধু ভুলসহ GPS মাপ দেখি। এতে দুটি সম্ভাবনা থাকে: (ক) **Emission probability**: এই রাস্তায় থাকলে এই GPS পড়ার সম্ভাবনা কত (রাস্তা থেকে যত দূরে, সম্ভাবনা তত কম)। (খ) **Transition probability**: এক রাস্তা থেকে পরের রাস্তায় যাওয়া কতটা সম্ভব (সংযুক্ত রাস্তা হলে বেশি)।
3. **Viterbi algorithm:** সব সম্ভাব্য রাস্তার ক্রমের মধ্যে সবচেয়ে সম্ভাব্য ক্রমটি দক্ষভাবে বের করে (dynamic programming দিয়ে, সব সম্ভাবনা আলাদা করে মাপার দরকার হয় না)।

4. **উদাহরণ:** ৩টি GPS বিন্দু আছে। প্রথম বিন্দু রাস্তা A ও B-র মাঝামাঝি। কিন্তু দ্বিতীয় ও তৃতীয় বিন্দু স্পষ্টভাবে A-র দিকে এবং A→C সংযুক্ত। Viterbi পুরো ক্রম দেখে সিদ্ধান্ত নেয়, প্রথম বিন্দুও আসলে A-তেই ছিল।
5. **শিক্ষকের প্রশ্ন:** "শুধু সবচেয়ে কাছের রাস্তা কেন নয়?" উত্তর: কাছের রাস্তা ভুল হতে পারে, যেমন সমান্তরাল রাস্তা বা ফ্লাইওভার। HMM পুরো যাত্রার ধারাবাহিকতা দেখে। স্লাইড ২২-এর Reference [2] (Newson & Krumm, 2009) এই বিষয়ের পরিচিত গবেষণা।

Sensor Fusion: একাধিক সেন্সরের তথ্য মিলিয়ে একটি ভালো অনুমান করা। যেমন GPS দুর্বল হলে IMU দিয়ে খানিকক্ষণ "অনুমানে" চলা (dead reckoning)।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **GPS ভুল হয় কেন?** ভবন (multipath), টানেল, আবহাওয়া ও দুর্বল স্যাটেলাইট সংকেতের কারণে।
2. **Map matching কেন দরকার?** GPS-এর ভুল সরিয়ে রাস্তার সঙ্গে ঠিক করে turn-by-turn নির্দেশনা সঠিক রাখতে।
3. **HMM-এ "hidden" কী?** আসল রাস্তা, যা আমরা সরাসরি দেখি না।
4. **Bottleneck?** কোটি ডিভাইসের জন্য একসঙ্গে real-time matching, তাই বেশিরভাগ কাজ ডিভাইসে বা edge-এ করা হতে পারে।
5. **ব্যাটারি?** ঘন ঘন GPS ব্যাটারি খায়, তাই সেন্সর মিলিয়ে ব্যবহার করা হয়।

F. Technical Clarification

- স্লাইডের পদ্ধতিগুলো (Kalman, HMM, Viterbi) পরিচিত ও গবেষণা-সমর্থিত, তবে Google Maps-এ ঠিক কোনটি চলে তা নিশ্চিত নয়। বলুন "এ ধরনের পদ্ধতি ব্যবহৃত হয়।"
- GPS-এর ৫–৩০ মিটার ভুল একটি সাধারণ আনুমানিক পরিসর, নির্দিষ্ট মান নয়।

G. Delivery Tips

- **গুরুত্ব:** "নীল বিন্দু যত সহজ দেখায়, তত সহজ নয়"।
- **ইঙ্গিত:** "Raw GPS" মানচিত্র ও "Matched map" পাশাপাশি দেখান। প্রথমে এলোমেলো বিন্দু, পরে রাস্তায় বসানো।
- **থামুন:** HMM বলার পর এক সেকেন্ড, আবার সহজ ভাষায় "মানে সবচেয়ে সম্ভাব্য রাস্তা" বলুন।
- **শেষ:** "নির্ভুল অবস্থান, মসৃণ navigation।"

স্লাইড ১২: Offline Maps & Data Synchronization

A. Slide Identification

- **Slide Number:** ১২
- **Slide Title:** Offline Maps & Data Synchronization
- **উদ্দেশ্য:** ইন্টারনেট ছাড়া কীভাবে মানচিত্র ও navigation চলে এবং পরে কীভাবে data হালনাগাদ হয়।

B. ব্যাখ্যা

চার ধাপ:

1. **Download Map Area:** ওয়াইফাইয়ে থাকাকালীন একটি এলাকা ডাউনলোড।
2. **Store Locally:** ডিভাইসে সংকুচিত ও এলাকাভিত্তিক সংরক্ষণ।
3. **Use Offline:** নেটওয়ার্ক ছাড়াই search ও navigation।
4. **Sync & Update:** ইন্টারনেট এলে শুধু পরিবর্তিত অংশ হালনাগাদ।

প্রযুক্তি: **On-device Map Storage** (সংকুচিত), **Data Compression** (tile/vector), **Incremental Updates** (শুধু যা বদলেছে), **Data Synchronization** (স্থানীয় ও cloud data মেলানো)।

Offline-এ পাওয়া যায়: জায়গা ও ঠিকানা খোঁজা, ড্রাইভিং directions ও ETA, রাস্তা ও স্থানের বিবরণ।

Incremental Update: পুরো মানচিত্র আবার ডাউনলোড না করে, শুধু বদলানো অংশ (যেমন একটি নতুন রাস্তা) ডাউনলোড। উদাহরণ: ফোনের অ্যাপ আপডেটে যেমন "delta update" থাকে।

C. Presentation Speech (প্রায় ১.৫ মিনিট)

> ধরুন আপনি পাহাড়ে বা গ্রামে যাচ্ছেন, যেখানে নেটওয়ার্ক নেই। তখনও Google Maps আপনাকে পথ দেখাতে পারে। এটা সম্ভব হয় offline maps সুবিধার কারণে।
>
> প্রক্রিয়াটি সহজ। প্রথমে ইন্টারনেট থাকা অবস্থায় আপনি একটি নির্দিষ্ট এলাকার মানচিত্র ডাউনলোড করেন। সেই data ফোনে সংকুচিত অবস্থায় জমা থাকে, যাতে কম জায়গা লাগে। পরে ইন্টারনেট না থাকলেও আপনি জায়গা খুঁজতে পারেন, driving directions ও ETA পেতে পারেন এবং রাস্তা ও স্থানের বিবরণ দেখতে পারেন।
>
> আবার যখন ইন্টারনেট ফিরে আসে, অ্যাপ নিজে থেকেই সিঙ্ক করে নেয়। এখানে পুরো মানচিত্র আবার ডাউনলোড হয় না, শুধু যা বদলেছে সেটুকু হালনাগাদ হয়। একে বলে incremental update। এতে মোবাইল data কম খরচ হয়।
>
> মূল কথা, স্থানীয় সংরক্ষণ আর cloud synchronization মিলিয়ে আমরা একটি নিরবচ্ছিন্ন মানচিত্র অভিজ্ঞতা পাই, এমনকি দুর্বল নেটওয়ার্কেও। এবার দেখি, বন্ধুর সঙ্গে live location শেয়ার কীভাবে হয়।

D. Additional Knowledge

Data Compression: ডেটার আকার ছোট করা। Vector tile সাধারণত raster ছবির চেয়ে অনেক ছোট, তাই offline-এ উপযোগী।

Synchronization ও Conflict: একই data ডিভাইসে ও cloud-এ আলাদাভাবে বদলাতে পারে। তখন কোনটি রাখা হবে তা ঠিক করার নিয়ম লাগে (যেমন "শেষ পরিবর্তনই জয়ী")। উদাহরণ: আপনি offline-এ একটি জায়গা সেভ করলেন, অন্য ডিভাইসে আরেকটি করলেন। Sync-এ দুটোই মিলিয়ে নেওয়া হয়।

Offline-এ কী কাজ করে না? Live traffic ও নতুন তথ্য, কারণ সেগুলোর জন্য সংযোগ লাগে। এটি বললে আপনার বোঝা প্রমাণিত হয়।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **Offline-এ traffic দেখা যায় কি?** না। Traffic live data, তাই ইন্টারনেট লাগে।
2. **Storage কীভাবে কম লাগে?** Compression, vector data ও এলাকাভিত্তিক ডাউনলোড।
3. **Offline map কত পুরোনো হতে পারে?** একটি সময়ের পর ডিভাইস সতর্ক করে বা আপডেট চায়। Sync-এ হালনাগাদ হয়।
4. **Bottleneck?** ডিভাইসের সীমিত storage ও বহু ব্যবহারকারীর একসঙ্গে আপডেট চাওয়া।

F. Technical Clarification

- স্লাইড সঠিক ও ধারণাগত। Offline-এ সব feature পাওয়া যায় না, তাই "supported features" বলুন (Notes-ও তাই বলে)।

G. Delivery Tips

- **গুরুত্ব:** "শুধু যা বদলেছে, তাই আপডেট"।
- **ইঙ্গিত:** Online phone ও Offline phone-এর ছবির দিকে।
- **থামুন:** পাহাড়ের উদাহরণের পর।
- **শেষ:** "স্থানীয় সংরক্ষণ + cloud sync"।

স্লাইড ১৩: Location Sharing & Real-Time Tracking

A. Slide Identification

- **Slide Number:** ১৩
- **Slide Title:** Location Sharing & Real-Time Tracking
- **উদ্দেশ্য:** লাইভ লোকেশন নিরাপদে ও কম delay-তে অন্যকে দেখানো।

B. ব্যাখ্যা

চার ধাপ (**Input → Processing → Output**):

1. **User Enables Sharing:** ব্যবহারকারী চালু করে।
2. **Send Live Location (to Google servers):** ডিভাইস নিয়মিত অবস্থান পাঠায়।
3. **Process & Update (real-time):** server গ্রহণ, যাচাই ও সংরক্ষণ করে।
4. **Show on Others' Maps:** অনুমোদিত ব্যক্তিরা চলমান অবস্থান দেখে।

প্রযুক্তি: **Location Services** (GPS/Wi-Fi/Cellular), **Real-Time Data Sync** (নিরাপদ ও কম latency), **Privacy & Access Control** (কে দেখবে, কতক্ষণ), **Scalable Cloud Infrastructure** (বহু live update সামলানো)।

Fan-out সমস্যা (স্লাইড ১৯-এও আছে): একজন ব্যবহারকারীর অবস্থান যদি ১০ জন দেখেন, একটি update ১০ জায়গায় পৌঁছাতে হয়। এই ছড়িয়ে দেওয়াকে বলে **fan-out**। লক্ষ লক্ষ ব্যবহারকারীর ক্ষেত্রে এটি বড় চ্যালেঞ্জ।

C. Presentation Speech (প্রায় ১.৫ মিনিট)

> Google Maps-এর একটি জনপ্রিয় সুবিধা হলো live location শেয়ার করা। যেমন, আপনি বাসা থেকে বেরিয়ে বন্ধুকে জানাতে পারেন আপনি কোথায় আছেন।
>
> প্রক্রিয়াটি এভাবে চলে। প্রথমে ব্যবহারকারী sharing চালু করেন। এরপর ফোন GPS ও অন্য পদ্ধতি দিয়ে অবস্থান নিয়ে তা নিরাপদভাবে Google-এর server-এ পাঠায়। Server সেই তথ্য প্রক্রিয়া করে এবং হালনাগাদ রাখে। শেষে যাদের অনুমতি দেওয়া আছে, তারা তাদের মানচিত্রে আমাদের চলমান অবস্থান দেখতে পায়।
>
> এখানে সবচেয়ে গুরুত্বপূর্ণ বিষয় হলো privacy ও access control। ব্যবহারকারী ঠিক করেন কে দেখবে এবং কতক্ষণ দেখবে। আর প্রযুক্তিগত দিক থেকে চ্যালেঞ্জ হলো, লক্ষ লক্ষ মানুষ একসঙ্গে লাইভ update পাঠালে system যেন কম delay-তে সবাইকে সেই update পৌঁছে দিতে পারে। এজন্য scalable cloud infrastructure দরকার।
>
> মূল কথা, real-time location service, cloud synchronization আর privacy control একসঙ্গে মিললে নির্ভরযোগ্য location sharing সম্ভব। এবার দেখি, ঠিকানা থেকে স্থানাঙ্ক কীভাবে বের হয়।

D. Additional Knowledge

Real-Time Sync কীভাবে? সাধারণত long-lived connection (যেমন WebSocket, বা মোবাইল push) ও Pub/Sub দিয়ে। Pub/Sub এ প্রকাশকারী (publisher) বার্তা পাঠায়, গ্রাহক (subscriber) পায়। উদাহরণ: ইউটিউব চ্যানেলে নতুন ভিডিও এলে সাবস্ক্রাইবারদের নোটিফিকেশন যায়।

Battery ও Update Frequency: প্রতি সেকেন্ডে পাঠালে ব্যাটারি ও data দুটোই বেশি খরচ। তাই বেশি নড়াচড়ায় ঘন ঘন, স্থির থাকলে কম আপডেট পাঠানো হয়।

Privacy: ব্যবহারকারী যেকোনো সময় শেয়ারিং বন্ধ করতে পারেন। সময়সীমা দেওয়া যায়।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **Delay কীভাবে কম রাখা হয়?** স্থায়ী সংযোগ, কাছের server, দ্রুত in-memory data ও সীমিত update-এর হার।
2. **Privacy কীভাবে?** অনুমতি, সময়সীমা, যে কোনো সময় বন্ধ করার সুযোগ ও encryption।
3. **Bottleneck?** Fan-out ও কোটি live connection ধরে রাখা (স্লাইড ১৯)।
4. **Scale কীভাবে?** ব্যবহারকারীদের শার্ড করে একাধিক server-এ ভাগ ও Pub/Sub দিয়ে বিতরণ।
5. **আপডেট ফেল করলে?** পুনরায় চেষ্টা, শেষ জানা অবস্থান দেখানো ও সময়সূচক চিহ্ন।

F. Technical Clarification

- স্লাইডটি ধারণাগতভাবে সঠিক। "Google servers" কীভাবে ঠিক কাজ করে তা প্রকাশিত নয়।
- Notes সঠিক।

G. Delivery Tips

- **গুরুত্ব:** privacy ও access control ("কে দেখবে, কতক্ষণ")।
- **ইঙ্গিত:** চার ধাপ বাঁ থেকে ডানে।
- **থামুন:** privacy-র কথা বলার আগে, যেন শ্রোতারা গুরুত্ব অনুভব করে।
- **শেষ:** "real-time + cloud + privacy।"

স্লাইড ১৪: Geocoding & Reverse Geocoding

A. Slide Identification

- **Slide Number:** ১৪
- **Slide Title:** Geocoding & Reverse Geocoding
- **উদ্দেশ্য:** মানুষের পড়ার ঠিকানা ও কম্পিউটারের স্থানাঙ্কের মধ্যে রূপান্তর বোঝানো।

B. ব্যাখ্যা

- **Geocoding:** ঠিকানা (Address) → স্থানাঙ্ক (Coordinates)। যেমন "খুলনা বিশ্ববিদ্যালয়" → (২২.৮১°, ৮৯.৫৪°) (আনুমানিক)।

- **Reverse Geocoding:** স্থানাঙ্ক → ঠিকানা। যেমন মানচিত্রে আঙুল চেপে ধরলে নিচে ঠিকানা আসে।

Geocoding-এর ধাপ: (১) Text Matching & Normalization: বানান ও ফরম্যাট ঠিক করা ("Rd" = "Road")। (২) Search & Match: বিশাল ঠিকানা-ডেটায় খোঁজা (spatial index দিয়ে)। (৩) Ranking & Disambiguation: একাধিক মিল থাকলে সেরাটি বাছা। (৪) Coordinates ফেরত।

Reverse Geocoding-এর ধাপ: (১) স্থানাঙ্ক গ্রহণ। (২) **সবচেয়ে কাছের ঠিকানা** খোঁজা (nearest neighbor)। (৩) ঠিকানা ফেরত।

Disambiguation কী? একাধিক মিলের মধ্যে সঠিকটি বাছা। "Springfield" নামে অনেক শহর আছে। ব্যবহারকারীর অবস্থান ও প্রসঙ্গ দেখে বাছা হয়।

C. Presentation Speech (প্রায় ১.৫ মিনিট)

> এই স্লাইডে আমরা দেখব, কম্পিউটার কীভাবে মানুষের লেখা ঠিকানা আর মানচিত্রের স্থানাঙ্কের মধ্যে আসা-যাওয়া করে। আমরা ঠিকানা বুঝি, কিন্তু কম্পিউটার বোঝে latitude আর longitude।
>
> Geocoding হলো ঠিকানা থেকে স্থানাঙ্ক বের করা। ব্যবহারকারী একটি ঠিকানা লিখলে system প্রথমে বানান আর ফরম্যাট ঠিক করে, যেমন সংক্ষিপ্ত রূপকে পূর্ণ রূপে নেয়। তারপর বিশাল ঠিকানার data-তে খুঁজে মিল বের করে। যদি একাধিক মিল থাকে, ranking দিয়ে সবচেয়ে ভালোটি বাছে। শেষে সেই জায়গার স্থানাঙ্ক ফেরত দেয়, আর মানচিত্রে পিন বসে।
>
> Reverse geocoding ঠিক উল্টো। আমরা মানচিত্রে একটি জায়গায় চাপ দিলে system সেই স্থানাঙ্কের সবচেয়ে কাছের ঠিকানা খুঁজে বের করে দেখায়।
>
> এই দুটি কাজ দ্রুত করতে লাগে বিশাল স্থান-তথ্য, R-tree বা QuadTree-র মতো spatial indexing, text matching আর ranking। মূল কথা, geocoding লেখা ঠিকানা আর ভৌগোলিক অবস্থানকে যুক্ত করে, যার ওপর search, navigation ও বহু location-based সেবা দাঁড়িয়ে আছে। এবার দেখব, এই সব সেবা কোন cloud কাঠামোর ওপর চলে।

D. Additional Knowledge

ঠিকানা কেন কঠিন? দেশভেদে ঠিকানার কাঠামো আলাদা। বাংলাদেশে অনেক জায়গায় ধারাবাহিক বাড়ি-নম্বর নেই। তাই ঠিকানা প্রায়ই landmark-নির্ভর ("মসজিদের পাশে")। এজন্য geocoding আরও কঠিন, যা শিক্ষকের সামনে একটি ভালো পর্যবেক্ষণ হবে।

Fuzzy Matching: বানান ভুল হলেও কাছাকাছি মিল পাওয়া (যেমন "Khulna" ও "Kulna")। সাধারণত edit distance বা n-gram দিয়ে।

Nearest Neighbor Search: Reverse geocoding আসলে "স্থানাঙ্কের সবচেয়ে কাছের ঠিকানা কোনটি" প্রশ্ন, যা R-tree/QuadTree (স্লাইড ৫) দিয়ে দ্রুত হয়।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **Geocoding ও Reverse Geocoding-এর পার্থক্য?** একটি ঠিকানা → স্থানাঙ্ক, অন্যটি স্থানাঙ্ক → ঠিকানা।
2. **Spatial index কেন লাগে?** কোটি কোটি ঠিকানা থেকে দ্রুত কাছেরটি খুঁজতে।
3. **ঠিকানা অস্পষ্ট হলে?** Ranking ও disambiguation, ব্যবহারকারীর অবস্থান ও প্রসঙ্গ দিয়ে।
4. **Bottleneck?** ভুল বা অসম্পূর্ণ ঠিকানা-ডেটা, আর জনপ্রিয় query-র চাপ (cache দিয়ে সামলানো যায়)।

F. Technical Clarification

- স্লাইড সঠিক ও ধারণাগত। কোনো ভুল তথ্য পাইনি।
- Notes-এর শেষ লাইনে "cloud infrastructure"-এ যাওয়ার transition ভালো, তাই এটি ব্যবহার করতে পারেন।

G. Delivery Tips

- **গুরুত্ব:** দুটির দিক উল্টো, এটি পরিষ্কার করা।
- **ইঙ্গিত:** Geocoding (ঠিকানা → স্থানাঙ্ক) আর Reverse (স্থানাঙ্ক → ঠিকানা) দুই সারিতে আলাদা হাত রাখুন।
- **থামুন:** দুই ধরনের পার্থক্য বলার পর।
- **শেষ:** "লেখা ঠিকানা ও ভৌগোলিক অবস্থানের সেতু।"

স্লাইড ১৫: System Architecture & Scalability

A. Slide Identification

- **Slide Number:** ১৫
- **Slide Title:** System Architecture & Scalability
- **উদ্দেশ্য:** পুরো system কোন cloud কাঠামোর ওপর দাঁড়িয়ে এবং কীভাবে scale করে।

B. ব্যাখ্যা

প্রবাহ: Users (Android, iOS, Web) → **Load Balancer** → **Microservices** (Map & Tile, Places & Search, Routing & Traffic, Geocoding) → **Caching & CDN** → **Data & Storage Layer** (Cache, Map Data, Places Data, Traffic Data; distributed, replicated, cached)।

Key Technologies: Google Cloud Infrastructure (বিশ্বব্যাপী, high availability), Load Balancer (ট্রাফিক বণ্টন), **Kubernetes** (service চালনা ও scale), Distributed Storage (**Spanner, Bigtable**), Microservices, Caching & CDN।

Scalability কী? চাহিদা বাড়লে system-এর সক্ষমতা বাড়ানোর ক্ষমতা। দুই ধরন: **Vertical** (একটি মেশিন শক্তিশালী করা, সীমা আছে) ও **Horizontal** (আরও মেশিন যোগ করা, Maps-এর মতো system-এ এটিই ব্যবহৃত)।

C. Presentation Speech (প্রায় ২ মিনিট)

> এতক্ষণ আমরা আলাদা আলাদা সেবা দেখেছি। এবার দেখব সব সেবা কোন অবকাঠামোর ওপর চলে এবং এত মানুষকে কীভাবে সামলায়।
>
> ব্যবহারকারীর request প্রথমে যায় একটি load balancer-এ। Load balancer হলো ট্রাফিক পুলিশের মতো। সে আসা request-গুলো একাধিক server-এ ভাগ করে দেয়, যাতে কোনো একটিতে অতিরিক্ত চাপ না পড়ে। তারপর request পৌঁছায় microservices-এ, যেখানে আলাদা আলাদা service আছে: map ও tile, place ও search, routing ও traffic, আর geocoding। প্রতিটি service একটি নির্দিষ্ট কাজ করে।
>
> এই service-গুলো চালায় ও scale করে Kubernetes-এর মতো container-ভিত্তিক ব্যবস্থা, ব্যবহারে চাপ বাড়লে নিজে থেকে আরও কপি চালু করে। এদের নিচে আছে distributed storage, যেমন Spanner বা Bigtable, যেখানে map, place ও traffic data জমা থাকে, একাধিক জায়গায় কপিসহ। আর caching ও CDN এই পুরো প্রক্রিয়াকে দ্রুত করে, কারণ জনপ্রিয় data কাছের server থেকেই পাওয়া যায়।
>
> এই কাঠামোর লক্ষ্য তিনটি: high availability, fault tolerance আর automatic scaling। মূল কথা, distributed cloud architecture আছে বলেই এত বড় বিশ্বব্যাপী সেবা নির্ভরযোগ্যভাবে চালানো যায়। এবার দেখি, request-টি কীভাবে gateway দিয়ে ঢোকে।

D. Additional Knowledge (বিস্তারিত)

Load Balancer

1. **কী?** আসা request একাধিক server-এ ভাগ করার ব্যবস্থা। 2. **কেন?** একটি server সব সামলাতে পারে না, আর সেটি নষ্ট হলে পুরো system বন্ধ হয়ে যায়। 3. **কীভাবে?** Round robin (পালা করে), least connections (যার চাপ কম) ইত্যাদি। অসুস্থ server-কে **health check** দিয়ে বাদ দেয়। 4. **Maps-এ:** প্রবেশপথে ও service-গুলোর মাঝে। 5. **উদাহরণ:** ব্যাংকে টোকেন-সিস্টেম, যে কাউন্টার ফাঁকা সেখানে পাঠানো। 6. **প্রশ্ন:** Load balancer নিজেই bottleneck হলে? তাকেও একাধিক রাখা হয়।

Kubernetes

1. **কী?** Container চালানো ও পরিচালনার ব্যবস্থা (orchestration)। **Container** মানে অ্যাপ ও তার দরকারি সব জিনিস একটি প্যাকেজে (যেমন Docker)। 2. **কেন?** শত শত service কীভাবে চালু, scale ও মেরামত হবে তা হাতে করা অসম্ভব। 3. **কীভাবে?** আপনি বলেন "এই service-এর ৫টি কপি চাই"। কোনোটি নষ্ট হলে সে আপনি নতুনটি চালু করে (**self-healing**), চাপ বাড়লে কপি বাড়ায় (**autoscaling**)। 4. **Maps-এ:** স্লাইড অনুযায়ী service চালনা ও scale। 5. **উদাহরণ:** ম্যানেজার যে জানে কোন বিভাগে কতজন কর্মী লাগবে এবং অসুস্থ হলে বদলি পাঠায়। 6. **প্রশ্ন:** "Google কি Kubernetes ব্যবহার করে?" Kubernetes আসলে Google-এর অভ্যন্তরীণ Borg-এর ধারণা থেকে তৈরি। তবে Maps ঠিক কী ব্যবহার করে তা নিশ্চিত নয়।

Spanner

1. **কী?** Google-এর বিশ্বব্যাপী distributed relational database (Reference [7])। 2. **কেন?** একই সঙ্গে বিশাল scale ও শক্তিশালী **consistency** (সব জায়গায় একই data দেখা) দেয়, যা সাধারণত কঠিন। 3. **কীভাবে?** Data অনেক ভাগে ও অঞ্চলে ছড়ানো ও কপি করা। বিশেষ ঘড়ি (**TrueTime**) ব্যবহার করে বিশ্বজুড়ে লেনদেনের ক্রম ঠিক রাখে। 4. **Maps-এ:** স্লাইডের উদাহরণ হিসেবে। 5. **উদাহরণ:** একই ব্যাংকের হিসাব বিশ্বের যেকোনো শাখায় একই ব্যালেন্স দেখাচ্ছে। 6. **প্রশ্ন:** CAP theorem? Spanner সাধারণত consistency-কে গুরুত্ব দেয়, আর খুব উচ্চ availability-ও দেয়, তবে এটি নেটওয়ার্ক বিভাজনে প্রয়োজনে consistency রাখে।

Bigtable

1. **কী?** Google-এর wide-column NoSQL distributed storage (Reference [6])। 2. **কেন?** বিপুল পরিমাণ data দ্রুত পড়া ও লেখার জন্য। 3. **কীভাবে?** Row key দিয়ে data সাজানো, অনেক মেশিনে ছড়ানো। 4. **Maps-এ:** traffic বা tile জাতীয় বিপুল, দ্রুত পরিবর্তনশীল data সংরক্ষণে উপযোগী (উদাহরণ)। 5. **উদাহরণ:** বিশাল গুদাম যেখানে প্রতিটি আইটেমের ঠিকানা (row key) আছে। 6. **প্রশ্ন:** Spanner বনাম Bigtable? Spanner relational ও transaction-সমর্থিত, Bigtable সহজ কিন্তু অতি দ্রুত ও বিশাল scale-এর NoSQL।

Microservices (বিস্তারিত স্লাইড ১৬-তে)।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **Horizontal ও Vertical scaling?** Horizontal = আরও মেশিন যোগ, Vertical = একটি মেশিন শক্তিশালী করা। বিশাল system-এ horizontal দরকার।
2. **High availability কীভাবে?** Redundancy, multi-region, load balancing ও failover।
3. **Kubernetes কেন?** Service চালনা, scale ও self-healing স্বয়ংক্রিয় করতে।
4. **Bottleneck?** Data layer, কারণ সব service শেষে সেখানে যায়। তাই cache ও replication।
5. **অন্য database ব্যবহার করা যেত কি?** হ্যাঁ, যেমন Cassandra, DynamoDB, তবে Spanner/Bigtable-এর মতো scale ও consistency-র চাহিদা থাকলে এগুলোই উপযোগী।

F. Technical Clarification

- স্লাইডটি reference architecture। "Microservices (on Google Cloud)" ও "Google Cloud Infrastructure" বোঝায় public cloud ব্যবহার। Google Maps ভেতরে Google-এর নিজস্ব অবকাঠামোয় চলে, এটি নিশ্চিত নয়।
- Spanner ও Bigtable বাস্তব Google প্রযুক্তি, কিন্তু Maps-এ ঠিক কোথায় ব্যবহৃত, তা প্রকাশিত নয়।

G. Delivery Tips

- **গুরুত্ব:** load balancer = ট্রাফিক পুলিশ, আর horizontal scaling।
- **ইঙ্গিত:** ওপর থেকে নিচে request-এর যাত্রা আঙুল দিয়ে দেখান।
- **থামুন:** Kubernetes ও Spanner-এর নাম বলার সময়, প্রতিটি সংক্ষেপে ব্যাখ্যা করুন।
- **শেষ:** "availability, fault tolerance, auto scaling।"

স্লাইড ১৬: API Gateway, Microservices & Communication

A. Slide Identification

- **Slide Number:** ১৬
- **Slide Title:** API Gateway, Microservices & Communication
- **উদ্দেশ্য:** request কীভাবে gateway দিয়ে ঢুকে সঠিক microservice-এ পৌঁছায় এবং service-গুলো কীভাবে নিজেদের মধ্যে কথা বলে।

B. ব্যাখ্যা

প্রবাহ (ওপর থেকে নিচে): Clients (Android, iOS, Web, Third-Party Apps) → **API Gateway** (single entry point) → **Microservices** (Map, Places, Routing, Traffic, Geocoding) → **Data Layer** (Map, Places, Traffic, User Data)।

API Gateway-এর চারটি কাজ: Request routing; Authentication & authorization; Rate limiting & quota; Monitoring & analytics।

Key Technologies: API Gateway, Microservices, **Service Communication** (REST, gRPC, Pub/Sub/event-driven), **Authentication & Security** (OAuth 2.0, API keys, IAM), **Observability** (logging, metrics, tracing)।

উদাহরণ: আপনি "Dhaka to Khulna directions" চাইলে gateway token যাচাই করে, রেট-লিমিট দেখে, request Routing Service-এ পাঠায়। Routing Service প্রয়োজনে Traffic Service ও Map Service-কে ডাকে।

C. Presentation Speech (প্রায় ২ মিনিট)

> এই স্লাইডে আমরা দেখব, একটি request কীভাবে system-এর ভেতরে ঢোকে এবং সঠিক জায়গায় পৌঁছায়। ক্লায়েন্ট হতে পারে Android, iOS, ওয়েব বা এমনকি তৃতীয় পক্ষের অ্যাপ।
>
> প্রতিটি request প্রথমে যায় API Gateway-তে। Gateway হলো একটি একক প্রবেশপথ, অনেকটা বিশ্ববিদ্যালয়ের মূল গেটের মতো। সে চারটি কাজ করে। প্রথমত, request কোন service-এ যাবে তা ঠিক করে, যাকে বলে routing। দ্বিতীয়ত, ব্যবহারকারী কে এবং তার অনুমতি আছে কি না তা দেখে। তৃতীয়ত, rate limiting ও quota ঠিক রাখে, যাতে কেউ অতিরিক্ত request পাঠিয়ে system অচল করতে না পারে। চতুর্থত, monitoring ও analytics করে।
>
> এরপর request যায় microservices-এ। Microservices মানে একটি বড় অ্যাপকে ছোট ছোট স্বাধীন service-এ ভাগ করা: Map, Places, Routing, Traffic ও Geocoding। এর বড় সুবিধা হলো, প্রতিটি service আলাদাভাবে তৈরি, deploy ও scale করা যায়। যেমন, সন্ধ্যায় routing-এর চাপ বাড়লে শুধু routing service বাড়ানো যায়, পুরো system নয়।
>
> এই service-গুলো নিজেদের মধ্যে কথা বলে REST, gRPC বা Pub/Sub দিয়ে। আর নিরাপত্তায় আছে OAuth 2.0, API key ও IAM। সঙ্গে observability, অর্থাৎ logging, metrics ও tracing, যা দিয়ে বোঝা যায় কোথায় সমস্যা। মূল কথা, API Gateway আর microservices জটিল distributed system-কে সহজে পরিচালনা, নিরাপদ ও scale করার সুযোগ দেয়। এবার দেখব, এই system কীভাবে দ্রুত থাকে caching দিয়ে।

D. Additional Knowledge (বিস্তারিত)

API Gateway

1. **কী?** সব client request-এর একক প্রবেশপথ। 2. **কেন?** নিরাপত্তা, নিয়ন্ত্রণ ও নজরদারি এক জায়গায় রাখতে। 3. **কীভাবে?** request আসে → token যাচাই → quota দেখা → সঠিক service-এ পাঠানো → উত্তর ফেরত। 4. **Maps-এ:** Maps Platform-এ API key দিয়ে ব্যবহারকারীর পরিচয় ও ব্যবহার সীমা নিয়ন্ত্রণ। 5. **উদাহরণ:** হাসপাতালের রিসেপশন। 6. **প্রশ্ন:** Gateway single point of failure? হতে পারে, তাই একাধিক instance চালানো হয়।

Microservices বনাম Monolith

- **Monolith:** পুরো অ্যাপ একটিই বড় প্রোগ্রাম। শুরুতে সহজ, কিন্তু একটি অংশ ভাঙলে সব ভাঙতে পারে, আর scale করতে হলে পুরোটা।
- **Microservices:** স্বাধীন ছোট service। সুবিধা: আলাদা scale, আলাদা deploy, একটি ভাঙলে অন্যগুলো বাঁচে। অসুবিধা: নেটওয়ার্কে যোগাযোগ বাড়ে, জটিলতা ও debug কঠিন।
- **উদাহরণ:** একটি বড় রান্নাঘরে একজন সব রাঁধলে (monolith) আর আলাদা সেকশন (microservices)।

REST বনাম gRPC বনাম Pub/Sub

- **REST:** HTTP ও JSON দিয়ে সহজ ও পরিচিত, সাধারণত বাইরের client-এর জন্য।
- **gRPC:** দ্রুত ও কম আকারের binary protocol। service-থেকে-service যোগাযোগে ভালো।
- **Pub/Sub:** asynchronous (সাথে সাথে উত্তরের অপেক্ষা না করা)। একটি service ঘটনা প্রকাশ করে, আগ্রহীরা পায়। যেমন, একটি দুর্ঘটনার ঘটনা Traffic, Routing ও Alert, তিন service-ই পেতে পারে।

Rate Limiting: একজন ব্যবহারকারী নির্দিষ্ট সময়ে কত request পাঠাতে পারবে তার সীমা। উদাহরণ: প্রতি মিনিটে ১০০টি। উদ্দেশ্য অপব্যবহার ও DDoS ঠেকানো।

Observability: Logging (কী ঘটল), Metrics (সংখ্যা, যেমন কত সেকেন্ড লাগল), Tracing (একটি request কোন কোন service ঘুরে এল)। উদাহরণ: পার্সেল ট্র্যাকিং।

API Key বনাম OAuth বনাম IAM: API key সহজ পরিচয়পত্র (কোন অ্যাপ), OAuth ব্যবহারকারীর পক্ষে অনুমতি (স্লাইড ৪), IAM (Identity and Access Management) কে কোন সম্পদে কী করতে পারবে তা নিয়ন্ত্রণ।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **API Gateway কেন দরকার?** নিরাপত্তা, routing, rate limiting ও monitoring এক জায়গায়, যাতে প্রতিটি service-কে আলাদাভাবে করতে না হয়।
2. **Microservices-এর অসুবিধা?** যোগাযোগ-জটিলতা, নেটওয়ার্ক delay, debug কঠিন।
3. **REST না gRPC?** বাইরের জন্য REST, ভেতরের দ্রুত যোগাযোগে gRPC।
4. **একটি service ধীর হলে?** Timeout, retry ও circuit breaker (স্লাইড ১৯)।
5. **Bottleneck?** Gateway ও service-গুলোর মধ্যে অতিরিক্ত নেটওয়ার্ক কল। Cache ও async যোগাযোগ কমায়।

F. Technical Clarification

- "API Gateway (e.g., Google Cloud API Gateway)" একটি **Google Cloud-এর গ্রাহকদের জন্য product**। Google Maps-এর অভ্যন্তরীণ প্রবেশপথ ঠিক এটিই, তা নিশ্চিত নয়। এটিকে উদাহরণ বলুন।
- Notes ঠিক আছে।

G. Delivery Tips

- **গুরুত্ব:** Gateway-র চার কাজ ও Microservices-এর "আলাদা scale" সুবিধা।
- **ইঙ্গিত:** ওপর (Clients) থেকে নিচে (Data Layer) আঙুল নামান।
- **থামুন:** বিশ্ববিদ্যালয়ের গেটের উপমার পর।
- **শেষ:** "জটিল system সহজে পরিচালনা, নিরাপদ ও scale।"

স্লাইড ১৭: Caching Architecture & CDN Optimization

A. Slide Identification

- **Slide Number:** ১৭
- **Slide Title:** Caching Architecture & CDN Optimization
- **উদ্দেশ্য:** map লোড ও পারফরম্যান্স দ্রুত করার কৌশল।

B. ব্যাখ্যা

প্রবাহ: Users → **CDN** (global edge servers) → **Caching Layer** (in-memory ও distributed) → **Backend Services**।

Cache Hit/Miss: request যেখানে প্রথম উত্তর পায় সেখানেই শেষ, এটিই লক্ষ্য। CDN-এ পেলে hit, না পেলে cache layer, তাতেও না থাকলে backend।

Caching Strategy: Tile Caching (tile ও ছবি), Data Caching (জায়গা, রিভিউ, ফলাফল), Cache Policies (**TTL**, eviction, refresh)।

Policies: TTL (Time To Live) মানে cache-এর data কতক্ষণ বৈধ। **Eviction** মানে জায়গা ভরে গেলে কোন data সরাবে (যেমন **LRU**, Least Recently Used, সবচেয়ে অনেকদিন ব্যবহার না হওয়াটি)। **Invalidation** মানে data বদলালে পুরোনো কপি বাতিল করা।

C. Presentation Speech (প্রায় ১.৫ মিনিট)

> মানচিত্র দ্রুত লোড হওয়ার পেছনে সবচেয়ে বড় ভূমিকা রাখে caching আর CDN। Caching মানে একবার আনা data কাছাকাছি কোথাও রেখে দেওয়া, যাতে পরের বার আবার দূর থেকে আনতে না হয়।
>
> এখানে request-এর পথ এমন: ব্যবহারকারী থেকে প্রথমে যায় CDN-এ, যা বিশ্বজুড়ে ছড়ানো edge server-এর নেটওয়ার্ক। সেখানে data পেলে সঙ্গে সঙ্গে উত্তর। না পেলে যায় caching layer-এ, যেটি মেমোরিতে থাকে বলে খুব দ্রুত, যেমন Redis বা Memcached। সেখানেও না পেলে তবেই যায় backend service-এ। অর্থাৎ বেশিরভাগ request পথেই শেষ হয়ে যায়।
>
> আমরা তিন ধরনের জিনিস cache করি: map tile ও ছবি, জায়গা ও রিভিউয়ের data, আর সার্চের ফলাফল। কিন্তু cache-এ পুরোনো data থাকলে সমস্যা, তাই আছে cache policy: TTL, মানে কতক্ষণ বৈধ; eviction, মানে জায়গা ভরলে কী সরবে; আর refresh বা invalidation, মানে data বদলালে নতুন করা।
>
> ফলাফলে network delay, database-এর চাপ ও খরচ কমে, আর ব্যবহারকারী দ্রুত সাড়া পায়। মূল কথা, caching ও CDN বিশ্বস্তরের map সেবাকে দ্রুত, দক্ষ ও scalable করে। এবার দেখি, এত ব্যক্তিগত data কীভাবে সুরক্ষিত থাকে।

D. Additional Knowledge

Cache vs CDN vs Database: Database স্থায়ী ও ধীর, Cache অস্থায়ী ও দ্রুত (সাধারণত RAM), CDN ভৌগোলিকভাবে ছড়ানো cache।

Cache Invalidation কেন কঠিন? একটি বিখ্যাত সমস্যা: data বদলালে কোন কোন কপি বাতিল করতে হবে তা ঠিক করা কঠিন। বেশি সময় রাখলে পুরোনো data, কম রাখলে সুবিধা কম। এটিই স্লাইড ১৯-এর "Freshness vs Cost" trade-off।

Memcached বনাম Redis: Memcached সহজ, কেবল key-value cache। Redis আরও বেশি data type ও features (স্লাইড ৫) দেয়।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **Caching কী কী সুবিধা দেয়?** দ্রুত সাড়া, কম database চাপ, কম খরচ।
2. **Cache-এ পুরোনো data থাকলে?** TTL ও invalidation দিয়ে সামলানো।
3. **CDN কীভাবে latency কমায়?** ব্যবহারকারীর কাছের edge server থেকে data দিয়ে।
4. **Bottleneck?** জনপ্রিয় item-এ অতি চাপ (**hot key**), বা একসঙ্গে অনেক cache miss (**cache stampede**)।
5. **সব data কি cache করা যায়?** না। ব্যক্তিগত বা দ্রুত বদলানো data (যেমন live location) বেশি cache করা যায় না।

F. Technical Clarification

- ⚠️ **ভুল শ্রেণিবিভাগ:** "Distributed Cache (e.g., Spanner, Bigtable with cache)"। Spanner ও Bigtable **database/storage**, cache নয়। বলুন: "Distributed cache-এর উদাহরণ Redis বা Memcached cluster, আর Spanner/Bigtable হলো persistent storage।"
- "Google Cloud CDN" আছে, তবে স্লাইড ৭-এর "Cloudflare"-এর সঙ্গে অসঙ্গতি, দুটিই উদাহরণ বলুন।

G. Delivery Tips

- **গুরুত্ব:** Hit/Miss-এর প্রবাহ ও TTL।
- **ইঙ্গিত:** Users → CDN → Cache → Backend তীর ধরে দেখান।
- **থামুন:** "বেশিরভাগ request পথেই শেষ" বাক্যের পর।
- **শেষ:** "কম delay, কম চাপ, কম খরচ।"

স্লাইড ১৮: Security, Authentication & Data Privacy

A. Slide Identification

- **Slide Number:** ১৮
- **Slide Title:** Security, Authentication & Data Privacy
- **উদ্দেশ্য:** অবস্থান-তথ্যের মতো সংবেদনশীল data ও অ্যাকাউন্টের সুরক্ষা।

B. ব্যাখ্যা

প্রবাহ: User → **Authentication** (Google Account, OAuth 2.0) → **Secure Access** (HTTPS/TLS, API Gateway) → **Google Maps Services**।

প্রধান ব্যবস্থা:

- **Authentication & Authorization:** তুমি কে (Authentication) এবং কী করতে পারো (Authorization)। Role-based access মানে ভূমিকা অনুযায়ী অনুমতি।
- **Data Encryption:** চলার পথে (**in transit**, TLS/HTTPS) ও জমা অবস্থায় (**at rest**) এনক্রিপশন।
- **Privacy Controls:** location history, activity ও data মোছার সুযোগ।
- **Access Control & Safety:** অনুমতি, rate limiting, অপব্যবহার ও প্রতারণা শনাক্তকরণ।
- **Data Handling Principles:** Data minimization (যতটুকু দরকার ততটুকুই), User consent (সম্মতি), Transparency (স্বচ্ছতা), Regulatory compliance (যেমন GDPR)।

C. Presentation Speech (প্রায় ১.৫ মিনিট)

> Google Maps আমাদের অবস্থান, যাত্রা ও অভ্যাসের মতো অনেক সংবেদনশীল তথ্য সামলায়। তাই নিরাপত্তা ও গোপনীয়তা এখানে অত্যন্ত গুরুত্বপূর্ণ।
>
> প্রথমে আসে authentication, অর্থাৎ ব্যবহারকারী আসলে কে তা যাচাই। এটি Google Account ও OAuth 2.0 দিয়ে হয়। তারপর authorization, অর্থাৎ সে কোন কোন তথ্য দেখতে পারবে। এরপর যোগাযোগ সুরক্ষিত থাকে HTTPS ও TLS এনক্রিপশন দিয়ে, যাতে পথে কেউ data পড়তে না পারে। আর জমা থাকা data-ও এনক্রিপ্ট করা থাকে, যাকে বলে encryption at rest।
>
> ব্যবহারকারীর নিয়ন্ত্রণও গুরুত্বপূর্ণ। সে নিজের location history দেখতে, বন্ধ করতে বা মুছতে পারে। আর system-এর দিক থেকে আছে অনুমতি নিয়ন্ত্রণ, rate limiting এবং অপব্যবহার ও প্রতারণা শনাক্তকরণ। পুরো ব্যবস্থার ভিত্তি কয়েকটি নীতি: প্রয়োজনের বেশি data না নেওয়া, ব্যবহারকারীর সম্মতি, স্বচ্ছতা এবং GDPR-এর মতো আইন মানা।
>
> মূল কথা, শক্তিশালী authentication, encryption আর privacy control মিলেই ব্যবহারকারীর আস্থা বজায় থাকে। এবার দেখি, এই system-এর দুর্বলতা বা সীমাবদ্ধতাগুলো কোথায়।

D. Additional Knowledge

TLS/HTTPS: ব্রাউজার ও server-এর মধ্যে যোগাযোগ এনক্রিপ্ট করা। উদাহরণ: সিলবন্ধ খামে চিঠি পাঠানো, ডাকপিয়ন খুলে পড়তে পারে না।

Encryption in transit vs at rest: চলাচলের সময় (নেটওয়ার্কে) বনাম জমা থাকা অবস্থায় (ডিস্কে)। দুটোই দরকার। একটি তালা দেওয়া পথ, অন্যটি তালা দেওয়া সিন্দুক।

Authentication বনাম Authorization: প্রথমটি "তুমি কে", দ্বিতীয়টি "তুমি কী করতে পারো"। উদাহরণ: বিমানবন্দরে পাসপোর্ট (authentication), বোর্ডিং পাস (authorization)।

GDPR: ইউরোপের তথ্য-সুরক্ষা আইন। ব্যবহারকারীর সম্মতি, data মোছার অধিকার ইত্যাদি দাবি করে।

Data Minimization: শুধু প্রয়োজনীয় data সংগ্রহ। কম data থাকলে ফাঁসের ক্ষতিও কম।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **Authentication ও Authorization-এর পার্থক্য?** পরিচয় যাচাই বনাম অনুমতি নির্ধারণ।
2. **Encryption in transit ও at rest কী?** চলাচলে ও জমা অবস্থায় সুরক্ষা।
3. **Location privacy কীভাবে নিশ্চিত হয়?** ব্যবহারকারীর সম্মতি, নিয়ন্ত্রণ ও মোছার সুযোগ, তথ্য ন্যূনতম রাখা।
4. **Bottleneck বা ঝুঁকি?** Encryption-এ সামান্য গণনা-খরচ, আর অ্যাকাউন্ট হ্যাক বা API key চুরির ঝুঁকি।
5. **API key চুরি ঠেকাতে?** key-কে ডোমেইন/অ্যাপে সীমাবদ্ধ করা ও quota রাখা।

F. Technical Clarification

- স্লাইডটি সাধারণ নিরাপত্তা নীতির ওপর, ধারণাগতভাবে ঠিক। ভেতরের সুনির্দিষ্ট নিরাপত্তা-ব্যবস্থা Google প্রকাশ করেনি, তাই এটি সাধারণ কাঠামো বলুন।
- Notes সঠিক।

G. Delivery Tips

- **গুরুত্ব:** authentication বনাম authorization, আর in transit/at rest।
- **ইঙ্গিত:** ওপরের চার ধাপের তীর, তারপর নিচের "Data Handling Principles" সারি।
- **থামুন:** "সংবেদনশীল তথ্য" বলার পর।
- **শেষ:** "আস্থাই মূল সম্পদ।"

স্লাইড ১৯: Bottlenecks, Failures & Trade-offs

A. Slide Identification

- **Slide Number:** ১৯
- **Slide Title:** Bottlenecks, Failures & Trade-offs (Where the system strains and what we give up)
- **উদ্দেশ্য:** system-এর দুর্বল জায়গা, ব্যর্থতায় করণীয় এবং নকশার আপস (trade-off) দেখানো।

B. ব্যাখ্যা

তিনটি কলাম আছে।

১. Bottlenecks (চাপের জায়গা):

- **Hot tiles in busy cities at rush hour:** ব্যস্ত শহরের কিছু tile বারবার চাওয়া হয়, তাই ওই tile-সংলগ্ন server-এ অতিরিক্ত চাপ। সমাধান: CDN, বেশি কপি, cache।
- **Route search over very large road graphs:** বিশাল graph-এ সেরা পথ খোঁজা ধীর। সমাধান: preprocessing (Contraction Hierarchies), graph ভাগ, A*।
- **Fan-out of live location updates:** এক update অনেক জনের কাছে পৌঁছানো (স্লাইড ১৩)। সমাধান: Pub/Sub, sharding, update-হার সীমিত করা।

২. Failures and responses (ব্যর্থতা ও প্রতিক্রিয়া):

- **Region outage → fail over:** একটি অঞ্চল বন্ধ হলে অন্য অঞ্চলে সরানো।
- **Slow service → timeouts, retries, circuit breakers:** ধীর service-এর জন্য অপেক্ষার সীমা, আবার চেষ্টা, আর **circuit breaker**।
- **Bad data release → staged rollout & rollback:** ধাপে ধাপে ছাড়া এবং দরকারে ফেরত নেওয়া।

৩. Trade-offs (আপস):

- **Freshness vs Cost:** বেশি সময় cache করলে খরচ কম, কিন্তু data পুরোনো।
- **Accuracy vs Speed:** আনুমানিক route দ্রুত, কিন্তু সামান্য কম নিখুঁত।
- **Consistency vs Availability:** map পড়ার ক্ষেত্রে availability বেশি গুরুত্বপূর্ণ।

C. Presentation Speech (প্রায় ২ মিনিট)

> কোনো system নিখুঁত নয়। প্রতিটি বড় system-এ কিছু জায়গায় চাপ পড়ে, কিছু অংশ ব্যর্থ হয়, আর আমাদের কিছু জিনিস বিসর্জন দিতে হয়। এই স্লাইডে আমরা সেটাই খোলাখুলি দেখব।
>
> প্রথমে bottleneck। প্রথমটি হলো ব্যস্ত শহরের কিছু tile, যা অফিসের সময় বারবার চাওয়া হয়, ফলে চাপ পড়ে। দ্বিতীয়টি হলো খুব বড় রাস্তার graph-এ route খোঁজা। আর তৃতীয়টি হলো live location update অনেক মানুষের কাছে পৌঁছানো, যাকে বলে fan-out।
>
> তারপর ব্যর্থতা। যদি একটি পুরো অঞ্চল বন্ধ হয়ে যায়, system অন্য অঞ্চলে সরে যায়, যাকে বলে failover। কোনো service ধীর হলে আছে timeout, retry ও circuit breaker। Circuit breaker অনেকটা বাড়ির ফিউজের মতো: সমস্যাযুক্ত অংশে বারবার request না পাঠিয়ে সাময়িক থামায়, যাতে পুরো system ডুবে না যায়। আর ভুল data ছাড়া হয়ে গেলে ধাপে ধাপে rollout ও rollback দিয়ে ক্ষতি সীমিত রাখা হয়।
>
> সবশেষে trade-off। আমরা যত বেশি সময় cache করি, খরচ কমে কিন্তু data পুরোনো হয়। আনুমানিক route দ্রুত উত্তর দেয় কিন্তু একটু কম নিখুঁত হতে পারে। আর map পড়ার ক্ষেত্রে consistency-র চেয়ে availability বেশি গুরুত্বপূর্ণ। কারণ কয়েক সেকেন্ড পুরোনো মানচিত্র দেখানো ভালো, মানচিত্র একেবারে না দেখানোর চেয়ে। মূল কথা, ভালো design মানে সব সমস্যা মুছে ফেলা নয়, বরং বুঝে-শুনে আপস করা। এবার দেখি, এই মানচিত্রের data কীভাবে তৈরি ও হালনাগাদ হয়।

D. Additional Knowledge

Circuit Breaker: (১) কোনো service বারবার ব্যর্থ হলে তার কাছে request পাঠানো সাময়িক বন্ধ করা। (২) ব্যর্থতা যেন ছড়িয়ে না পড়ে (cascading failure)। (৩) তিন অবস্থা: Closed (স্বাভাবিক), Open (বন্ধ), Half-open (পরীক্ষামূলক)। (৪) উদাহরণ: বাড়ির MCB/ফিউজ।

Failover: প্রধান অংশ নষ্ট হলে স্বয়ংক্রিয়ভাবে বিকল্প অংশে সরে যাওয়া। উদাহরণ: জেনারেটর, বিদ্যুৎ গেলে স্বয়ংক্রিয়ভাবে চালু।

Retry-তে সতর্কতা: সবাই একসঙ্গে বারবার চেষ্টা করলে সমস্যা বাড়ে (retry storm)। তাই **exponential backoff** (প্রতিবার অপেক্ষা বাড়ানো) ও randomness ব্যবহার হয়।

CAP Theorem: বিতরিত system-এ নেটওয়ার্ক বিভাজনের (Partition) সময় Consistency ও Availability একসঙ্গে পুরো পাওয়া যায় না, একটি বেছে নিতে হয়। Maps-এর পড়ার ক্ষেত্রে সাধারণত Availability বেছে নেওয়া হয়।

Staged Rollout/Canary: নতুন সংস্করণ প্রথমে অল্প ব্যবহারকারীকে দেওয়া, ঠিক থাকলে ধীরে ধীরে সবাইকে।

Tail Latency: স্লাইড ২২-এর Reference [8] (The Tail at Scale)। বহু service একসঙ্গে ডাকলে সবচেয়ে ধীরটির জন্য পুরো উত্তর ধীর হয়ে যায়। এটি বিশাল system-এর একটি বড় সমস্যা।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **Hot tile সমস্যা কীভাবে সামলানো যায়?** CDN, বহু কপি, জনপ্রিয় tile আগে থেকে cache।
2. **Availability কেন consistency-র চেয়ে অগ্রাধিকার?** মানচিত্র সামান্য পুরোনো হলেও ব্যবহারযোগ্য, বন্ধ থাকলে নয়।
3. **Circuit breaker কী?** ব্যর্থ service-কে সাময়িক এড়িয়ে ব্যর্থতার ছড়ানো ঠেকানো।
4. **Region outage-এ কী হয়?** ট্রাফিক অন্য অঞ্চলে সরানো হয় (failover), data আগে থেকেই কপি করা থাকে।
5. **ভুল data ছাড়া হলে?** Staged rollout ধরিয়ে দেয়, rollback দিয়ে ফেরত।

F. Technical Clarification

- ⚠️ **Notes ভুল:** এই স্লাইডের Notes আসলে পুরো presentation-এর সমাপনী সারাংশ। এটি ব্যবহার করবেন না।
- স্লাইডের তথ্য সঠিক ও ধারণাগত। "Approximate routes" একটি design উদাহরণ।
- ক্রম: এই স্লাইডটি data pipeline (২০, ২১)-এর আগে এসেছে। প্রয়োজনে মুখে বলুন "architecture দেখার পর তার চাপ ও আপস দেখলাম, এবার data কীভাবে তৈরি হয় দেখি।"

G. Delivery Tips

- **গুরুত্ব:** circuit breaker ও availability-over-consistency।
- **ইঙ্গিত:** তিন কলাম বাঁ থেকে ডানে, প্রতিটিতে থামুন।
- **থামুন:** "কোনো system নিখুঁত নয়" বাক্যের পর। এটি শিক্ষকের ওপর ভালো প্রভাব ফেলে।
- **শেষ:** "বুঝে-শুনে আপস, এটাই ভালো design।"

স্লাইড ২০: Maps Data Pipeline & Data Processing

A. Slide Identification

- **Slide Number:** ২০
- **Slide Title:** Maps Data Pipeline & Data Processing
- **উদ্দেশ্য:** বাস্তব জগতের তথ্য কীভাবে সংগ্রহ, প্রক্রিয়া ও মানচিত্রে হালনাগাদ হয়।

B. ব্যাখ্যা

পাঁচ ধাপের pipeline (Input → Processing → Output):

1. **Data Sources:** স্যাটেলাইট ও aerial ছবি, Street View, ব্যবহারকারীর অবদান (রিভিউ, সম্পাদনা), traffic ও sensor data (GPS, মোবাইল), authoritative data (সরকার, সহযোগী)।
2. **Data Ingestion:** তথ্য গ্রহণ ও প্রক্রিয়ার জন্য প্রস্তুত।
3. **Data Processing (at scale):** মূল পদক্ষেপ: **Image Analysis**, **Change Detection**, **Data Fusion**।
4. **Data Validation & Enrichment: Verification** (স্বয়ংক্রিয় + মানুষের পর্যালোচনা) ও মান উন্নয়ন।
5. **Map Data Update:** rolling ও incremental হালনাগাদ।

Key Technologies: ML/AI (road/object detection), Large-Scale Data Processing (Apache Beam, Dataflow), BigQuery (geospatial analytics), Geospatial Databases (Spanner, Bigtable), Data Pipelines (ETL/stream processing)।

উদাহরণ: স্যাটেলাইট ছবিতে নতুন একটি সড়ক ধরা পড়ল (change detection)। GPS data-তেও সেই পথে গাড়ি চলছে (data fusion)। মানুষ বা নিয়ম যাচাই করল (verification)। মানচিত্রে নতুন সড়ক যোগ হলো (update)।

C. Presentation Speech (প্রায় ২ মিনিট)

> আমরা প্রায়ই ভাবি মানচিত্র একবার তৈরি হয়ে গেলেই কাজ শেষ। কিন্তু বাস্তবে নতুন রাস্তা তৈরি হয়, দোকান বদলায়, সেতু বন্ধ হয়। তাই মানচিত্রকে নিরন্তর হালনাগাদ রাখতে একটি বিশাল data pipeline লাগে।
>
> শুরুতে data আসে নানা উৎস থেকে: স্যাটেলাইট ও আকাশ থেকে তোলা ছবি, Street View-এর গাড়ির তোলা ছবি, ব্যবহারকারীর দেওয়া রিভিউ ও সম্পাদনা, GPS ও sensor data, এবং সরকার বা সহযোগী প্রতিষ্ঠানের নির্ভরযোগ্য data। সেগুলো ingestion ধাপে সংগ্রহ হয়ে processing ধাপে যায়।
>
> Processing-এর তিনটি মূল কাজ: প্রথমত, ML দিয়ে ছবি বিশ্লেষণ করে রাস্তা বা স্থান শনাক্ত করা। দ্বিতীয়ত, change detection, অর্থাৎ আগের সঙ্গে তুলনা করে কী বদলেছে তা ধরা। তৃতীয়ত, data fusion, অর্থাৎ একাধিক উৎসের তথ্য মিলিয়ে নিশ্চিত হওয়া। এই কাজের জন্য Apache Beam ও Dataflow-এর মতো বড় আকারের processing সরঞ্জাম আর BigQuery-এর মতো analytics ব্যবস্থা ব্যবহার করা যায়।
>
> এরপর validation: স্বয়ংক্রিয় ও মানুষের যাচাই। শেষে নিশ্চিত পরিবর্তনগুলো ধাপে ধাপে মানচিত্রে যুক্ত হয়। মূল কথা, Google Maps কোনো স্থির মানচিত্র নয়, এটি একটি ধারাবাহিকভাবে হালনাগাদ হওয়া geographic information system। এবার দেখি, AI এই কাজে কীভাবে সাহায্য করে।

D. Additional Knowledge

ETL ও Stream/Batch Processing: ETL = Extract (সংগ্রহ), Transform (রূপান্তর), Load (জমা)। **Batch** হলো জমা করে একসঙ্গে প্রক্রিয়া (যেমন রাতে স্যাটেলাইট ছবি)। **Stream** হলো আসামাত্র প্রক্রিয়া (যেমন traffic)। দুটোই লাগে।

Apache Beam / Dataflow: Beam একটি programming model যা একই কোডে batch ও stream দুটোই চালায়। Dataflow হলো Google Cloud-এর managed service যা Beam কোড চালায়। উদাহরণ: একটি কারখানার অ্যাসেম্বলি লাইন, যেখানে ছবি বা data ধাপে ধাপে প্রক্রিয়া হয়।

BigQuery: Google Cloud-এর বিশাল data বিশ্লেষণের গুদাম, SQL দিয়ে। এতে geospatial function আছে। উদাহরণ: "এই শহরে গত মাসে কোথায় সবচেয়ে বেশি ধীরগতি ছিল?"

Data Fusion: বিভিন্ন উৎস মিলিয়ে একটি নির্ভরযোগ্য সিদ্ধান্ত। উদাহরণ: একজন ব্যবহারকারী বলল দোকান বন্ধ। কিন্তু আরও ১০ জন একই বলল, আর স্যাটেলাইট ছবিও বদলেছে। তখন বিশ্বাসযোগ্যতা বাড়ে।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **Pipeline কেন দরকার?** বাস্তব জগৎ বদলায়, তাই মানচিত্র নিয়মিত হালনাগাদ লাগে।
2. **Data fusion কেন?** একক উৎস ভুল হতে পারে, একাধিক উৎস মিললে নির্ভরযোগ্যতা বাড়ে।
3. **Verification কেন মানুষও লাগে?** ML ভুল করে, বিশেষত অস্পষ্ট ছবি বা নতুন পরিস্থিতিতে, তাই মানুষের পর্যালোচনা।
4. **Bottleneck?** বিশাল ছবির ডেটা প্রক্রিয়া, ও ভুয়া ব্যবহারকারী-তথ্য ঠেকানো।
5. **Batch না stream?** দুটোই: ধীরগতির বিষয়ে batch, দ্রুত বদলানোয় stream।

F. Technical Clarification

- Apache Beam, Dataflow ও BigQuery বাস্তব Google Cloud সরঞ্জাম, তবে Google Maps-এর ভেতরের pipeline ঠিক এগুলোই কি না তা নিশ্চিত নয়। এগুলো উদাহরণ।
- Notes সঠিক।

G. Delivery Tips

- **গুরুত্ব:** "Google Maps স্থির নয়, নিরন্তর হালনাগাদ হওয়া", আর fusion ও verification।
- **ইঙ্গিত:** পাঁচ ধাপ বাঁ থেকে ডানে, নিচের পাঁচ উৎসে হাত রাখুন।
- **থামুন:** উৎস বলার পর, তারপর processing-এ যান।
- **শেষ:** "ধারাবাহিক হালনাগাদ, এটাই Maps-এর প্রাণ।"

স্লাইড ২১: AI, Computer Vision & Satellite Imagery

A. Slide Identification

- **Slide Number:** ২১
- **Slide Title:** AI, Computer Vision & Satellite Imagery
- **উদ্দেশ্য:** AI কীভাবে ছবি থেকে মানচিত্রের তথ্য বের করে।

B. ব্যাখ্যা

পাঁচ ধাপ (End-to-End): Image Collection (স্যাটেলাইট, aerial, Street View) → AI Analysis (computer vision, ML) → Feature Extraction (রাস্তা, ভবন, স্থান, ভূমি-ব্যবহার) → Validation (মানুষ ও স্বয়ংক্রিয়) → Map Update (ধারাবাহিক)।

Key Technologies: ML/Deep Learning (object detection, classification), Computer Vision (image analysis, **segmentation**), Satellite & Aerial Imagery (উচ্চ রেজুলিউশন), Street View (মাটি-থেকে ৩৬০° ছবি), Large-Scale Processing (Google Cloud, **TPU**)।

চারটি উদাহরণ: Road Detection (রাস্তা, লেন, মোড়), Building Detection (ভবনের সীমানা), Place Recognition (দোকান, ল্যান্ডমার্ক), Change Detection (নতুন নির্মাণ)।

সহজ পার্থক্য: Classification = ছবিটি কী ("এটি ভবন")। **Object Detection** = ছবির কোথায় কী (বাক্স দিয়ে)। **Segmentation** = প্রতিটি পিক্সেল কোন শ্রেণির (সবচেয়ে নিখুঁত, রাস্তা বা ভবনের সঠিক আকার)।

C. Presentation Speech (প্রায় ২ মিনিট)

> পুরো পৃথিবীর মানচিত্র মানুষ হাতে হাতে আঁকবে, এটা অসম্ভব। এখানেই কাজে আসে AI ও computer vision। Computer vision মানে কম্পিউটারকে ছবি বুঝতে শেখানো।
>
> প্রক্রিয়াটি শুরু হয় ছবি সংগ্রহ দিয়ে: স্যাটেলাইট, আকাশ থেকে তোলা ছবি আর Street View-এর মাটি-থেকে তোলা ৩৬০ ডিগ্রি ছবি। এরপর AI model সেই ছবি বিশ্লেষণ করে। সে ছবির মধ্যে রাস্তা, ভবন, দোকান ও ভূমির ব্যবহার শনাক্ত করে, যাকে বলে feature extraction।
>
> স্লাইডে চারটি উদাহরণ আছে। Road detection, যেখানে রাস্তা, লেন ও মোড় খোঁজা হয়। Building detection, যেখানে ভবনের সীমানা ও আকার বের করা হয়। Place recognition, যেখানে দোকান বা ল্যান্ডমার্ক শনাক্ত ও যাচাই করা হয়। আর change detection, যেখানে আগের ছবির সঙ্গে তুলনা করে নতুন নির্মাণ বা পরিবর্তন ধরা হয়।
>
> কিন্তু AI-এর উত্তর সরাসরি মানচিত্রে বসে না। আগে যায় validation, যেখানে স্বয়ংক্রিয় ও মানুষের যাচাই থাকে। তারপর ধারাবাহিকভাবে মানচিত্র হালনাগাদ হয়। এই বিশাল গণনার জন্য লাগে Google Cloud ও TPU-র মতো শক্তিশালী হার্ডওয়্যার। মূল কথা, AI বিপুল পরিমাণ ছবিকে কাঠামোবদ্ধ ভৌগোলিক data-তে রূপান্তর করে। এবার আমরা পুরো আলোচনা থেকে কী শিখলাম তা দেখব।

D. Additional Knowledge

Computer Vision ও Deep Learning (CNN): (১) *কী?* ছবি বোঝার AI শাখা। (২) *কেন?* হাতে ছবি দেখে মানচিত্র আঁকা অসম্ভব। (৩) *কীভাবে?* **CNN (Convolutional Neural Network)** ছবির ছোট ছোট অংশে ফিল্টার চালিয়ে প্রথমে ধার ও রেখা, পরে আকৃতি, শেষে "এটি রাস্তা" শেখে। (৪) *Maps-এ:* রাস্তা, ভবন, সাইনবোর্ড শনাক্তকরণ। (৫) *উদাহরণ:* শিশু যেমন অসংখ্য ছবি দেখে বিড়াল চিনতে শেখে। (৬) *প্রশ্ন:* Training data কোথা থেকে? মানুষ-চিহ্নিত (labeled) ছবি থেকে।

TPU (Tensor Processing Unit): Google-এর তৈরি বিশেষ chip, AI/ML গণনাকে দ্রুত করতে। সাধারণ CPU-র চেয়ে ML-এ অনেক দ্রুত।

Street View ও গোপনীয়তা: Street View ছবিতে মুখ ও গাড়ির নম্বর প্লেট স্বয়ংক্রিয়ভাবে ঝাপসা করা হয় (এটি Google-এর প্রকাশিত নীতি)। গোপনীয়তার প্রশ্নে এটি ভালো উত্তর।

ভুল শনাক্তকরণ: ছায়া, মেঘ বা অস্পষ্ট ছবি থেকে ভুল হতে পারে। এজন্য validation ধাপ জরুরি।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **AI কেন দরকার, মানুষ কেন নয়?** বিশ্বব্যাপী ছবির পরিমাণ এত বিশাল যে মানুষের একার পক্ষে অসম্ভব।
2. **Classification, Detection, Segmentation-এর পার্থক্য?** কী, কোথায় ও নিখুঁত আকার।
3. **AI ভুল করলে?** Validation: স্বয়ংক্রিয় ও মানুষের যাচাই।
4. **Bottleneck?** বিশাল ছবির প্রক্রিয়া-খরচ ও labeled data-র অভাব।

5. **ছবি ও গোপনীয়তা?** মুখ ও প্লেট ঝাপসা, নীতি অনুসরণ।

F. Technical Clarification

- স্লাইডটি সাধারণ ধারণা-নির্ভর। "Large-Scale Processing (Google Cloud, TPU)" ঠিক, তবে Maps-এ ঠিক কী মডেল বা hardware চলে, তা প্রকাশিত নয়।
- Notes সঠিক।

G. Delivery Tips

- **গুরুত্ব:** AI-র আউটপুট সরাসরি মানচিত্রে যায় না, validation হয়।
- **ইঙ্গিত:** চারটি ছবির উদাহরণে (road, building, place, change) একে একে আঙুল রাখুন।
- **থামুন:** "Computer vision মানে..." বলার পর।
- **শেষ:** "ছবি → তথ্য → মানচিত্র।"

স্লাইড ২২: Key Lessons & Sources (References)

A. Slide Identification

- **Slide Number:** ২২
- **Slide Title:** Key Lessons & Sources (What to take away, and where to read more)
- **উদ্দেশ্য:** আমাদের তথ্যের গবেষণা-ভিত্তি দেখানো এবং আরও পড়ার সুযোগ দেওয়া।

B. ব্যাখ্যা

স্লাইডে আটটি রেফারেন্স আছে। কোনটি আমাদের কোন স্লাইডের সঙ্গে মেলে:

- **[1] Derrow-Pinion et al. (2021): ETA Prediction with Graph Neural Networks in Google Maps (CIKM)** → স্লাইড ১০। এটি Google/DeepMind-এর গবেষণা, GNN দিয়ে ETA অনুমান। এই তালিকার সবচেয়ে সরাসরি "Google Maps"-সংক্রান্ত উৎস।
- **[2] Newson & Krumm (2009): Hidden Markov Map Matching Through Noise and Sparseness** → স্লাইড ১১। HMM দিয়ে map matching-এর ক্লাসিক গবেষণা।
- **[3] Geisberger et al. (2012): Contraction Hierarchies** → স্লাইড ৬ ও ১৯। বিশাল road graph-এ দ্রুত route খোঁজা।
- **[4] Hart, Nilsson & Raphael (1968): A* algorithm-এর মূল গবেষণাপত্র** → স্লাইড ৬।
- **[5] Bast et al. (2016): Route Planning in Transportation Networks** → স্লাইড ৬। route planning-এর পূর্ণ জরিপ।

- **[6] Chang et al. (2008): Bigtable** → স্লাইড ১৫, ২০।
- **[7] Corbett et al. (2013): Spanner** → স্লাইড ১৫, ২০।
- **[8] Dean & Barroso (2013): The Tail at Scale** → স্লাইড ১৯। বিশাল system-এর ধীর অংশের প্রভাব।

এই রেফারেন্সগুলো দেখায় যে আমাদের আলোচিত অনেক ধারণা প্রকাশিত গবেষণার ওপর দাঁড়িয়ে। তবে এগুলো Google Maps-এর সম্পূর্ণ architecture নয়।

C. Presentation Speech (প্রায় ৩০–৪৫ সেকেন্ড)

> আমাদের এই presentation-এর তথ্য শুধু কল্পনা থেকে নয়, বরং প্রকাশিত গবেষণার ওপর ভিত্তি করে। এই স্লাইডে আটটি রেফারেন্স আছে, যা আমরা আমাদের আলোচনার সঙ্গে মিলিয়ে ব্যবহার করেছি।
>
> যেমন, প্রথম রেফারেন্সটি Google Maps-এ Graph Neural Network দিয়ে ETA অনুমানের গবেষণা, যা ETA স্লাইডের সঙ্গে সম্পর্কিত। দ্বিতীয়টি HMM দিয়ে map matching, যা GPS স্লাইডের সঙ্গে মেলে। তৃতীয় থেকে পঞ্চম রেফারেন্স routing, A* ও Contraction Hierarchies নিয়ে। ষষ্ঠ ও সপ্তম রেফারেন্স Google-এর Bigtable ও Spanner, যা storage অংশে এসেছে। আর অষ্টমটি The Tail at Scale, যা বিশাল system-এর ধীরগতির সমস্যা নিয়ে।
>
> যাঁরা আরও গভীরে জানতে চান, তাঁরা এই উৎসগুলো পড়তে পারেন। এবার আমরা আমাদের আলোচনার মূল শিক্ষাগুলো দেখব।

D. Additional Knowledge

সময় থাকলে শুধু Reference [1], [2] ও [8] সম্পর্কে এক বাক্যে বলতে পারেন, বাকিদের নাম বলাই যথেষ্ট। প্রতিটির ব্যাখ্যা ওই ওই স্লাইডে দেওয়া আছে (A*: স্লাইড ৬, Spanner/Bigtable: স্লাইড ১৫, HMM: স্লাইড ১১)।

The Tail at Scale (সংক্ষেপে): একটি request যদি ১০০টি সার্ভারে যায়, আর প্রতিটির ১% ক্ষেত্রে ধীর হওয়ার সম্ভাবনা থাকে, তাহলে অন্তত একটি ধীর হওয়ার সম্ভাবনা অনেক বেড়ে যায় (প্রায় ৬৩%)। সমাধান: hedged request (একই request দুই জায়গায় পাঠানো, যে আগে উত্তর দেয় সেটি নেওয়া) ইত্যাদি। এটি জানলে শিক্ষক প্রভাবিত হবেন।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **এই রেফারেন্সগুলোর মধ্যে সবচেয়ে গুরুত্বপূর্ণ কোনটি?** [1], কারণ এটি সরাসরি Google Maps-এর ETA নিয়ে।
2. **Spanner ও Bigtable-এর গবেষণাপত্র কেন?** Google নিজেই এগুলো প্রকাশ করেছে, তাই বিশ্বস্ত তথ্যসূত্র।
3. **এই রেফারেন্স কি Maps-এর পুরো architecture দেয়?** না, শুধু আলাদা আলাদা অংশের ভিত্তি দেয়।

F. Technical Clarification

- ⚠️ **স্লাইডের শিরোনাম বিভ্রান্তিকর:** স্লাইড ২২-এর শিরোনাম "Key Lessons & Sources" হলেও শুধু Sources আছে। আসল "Key Lessons" আছে স্লাইড ২৩-এ। এটি মুখে একটু সামলে নিন।
- ⚠️ **Notes ভুল:** এই স্লাইডের Notes অপ্রাসঙ্গিক। ব্যবহার করবেন না।
- রেফারেন্সগুলো আসল ও সঠিক গবেষণাপত্র, তবে [3], [4], [5], [8] স্লাইডের মূল অংশে সরাসরি উল্লেখ নেই।

G. Delivery Tips

- **গুরুত্ব:** [1] ও [2], যেহেতু সরাসরি সম্পর্কিত।
- **ইঙ্গিত:** পুরো তালিকায় আঙুল না বুলিয়ে শুধু [1], [2], [6], [7]-এ ইঙ্গিত করুন।
- **থামুন:** এখানে দ্রুত যান, ৩০–৪৫ সেকেন্ডই যথেষ্ট।
- **শেষ:** "এবার মূল শিক্ষাগুলোতে আসি।"

স্লাইড ২৩: Key Lessons / Thank You

A. Slide Identification

- **Slide Number:** ২৩
- **Slide Title:** Key lessons / Thank You
- **উদ্দেশ্য:** মূল শিক্ষা সংক্ষেপে বলা এবং presentation শেষ করা।

B. ব্যাখ্যা

চারটি শিক্ষা:

1. **Cache first: most requests should never reach a service.** বেশিরভাগ request cache/CDN-এ শেষ হওয়া উচিত (স্লাইড ৭, ১৭)।
2. **Split into small services that scale independently.** ছোট ছোট স্বাধীন service (স্লাইড ১৫, ১৬)।
3. **Design for failure in every region.** প্রতিটি অঞ্চলের ব্যর্থতার জন্য আগে থেকে প্রস্তুত থাকা (স্লাইড ১৯)।
4. **Treat data freshness as a product choice with a cost.** data কতটা তাজা হবে তা একটি সচেতন সিদ্ধান্ত, যার খরচ আছে (স্লাইড ১৭, ১৯, ২০)।

সঙ্গে "Thank You", "Questions and discussion" ও "Google Maps system design"।

C. Presentation Speech (প্রায় ১.৫ মিনিট)

> আমরা আজ Google Maps-কে একটি system design-এর দৃষ্টিতে দেখলাম। শেষে চারটি মূল শিক্ষা আপনাদের সঙ্গে ভাগ করে নিতে চাই।
>
> প্রথমত, cache first। অর্থাৎ বেশিরভাগ request-ই যেন কখনো মূল service পর্যন্ত না পৌঁছায়, কারণ CDN বা cache থেকেই উত্তর পাওয়া যায়। দ্বিতীয়ত, system-কে ছোট ছোট service-এ ভাগ করতে হবে, যেন প্রতিটি আলাদাভাবে বাড়ানো বা বদলানো যায়। তৃতীয়ত, ব্যর্থতার জন্য আগে থেকে ডিজাইন করতে হবে, কারণ বিশাল system-এ কিছু না কিছু সবসময় ভাঙবে। আর চতুর্থত, data কতটা নতুন রাখব, সেটি একটি product-সিদ্ধান্ত, যার একটি মূল্য আছে।
>
> আমাদের উপস্থাপনা এখানেই শেষ। আপনাদের ধৈর্য ধরে শোনার জন্য ধন্যবাদ। আপনাদের কোনো প্রশ্ন বা মন্তব্য থাকলে আমরা আনন্দের সঙ্গে আলোচনা করব।

D. Additional Knowledge

এই চারটি শিক্ষা পুরো presentation-এর সারাংশ। প্রতিটির সঙ্গে একটি স্লাইড মিলিয়ে রাখুন, শিক্ষক যে কোনোটি নিয়ে প্রশ্ন করতে পারেন।

E. শিক্ষকের সম্ভাব্য প্রশ্ন

1. **"Cache first" মানে কি সব data cache?** না, শুধু বারবার চাওয়া ও কম বদলানো data। Live location বা ব্যক্তিগত data নয়।
2. **সবচেয়ে বড় শিক্ষা কোনটি?** আপনার মত দিতে পারেন, যেমন "design for failure", কারণ বিশাল system-এ ব্যর্থতা স্বাভাবিক।
3. **এই design-এর মূল ঝুঁকি কী?** জটিলতা, ও data-র ধারাবাহিকতা রক্ষা।

F. Technical Clarification

- স্লাইডে কোনো ভুল তথ্য নেই।
- Notes: আপনার স্লাইড ২২-এর মতোই Notes সম্ভবত একই সমাপনী সারাংশ দেখাচ্ছে। এই গাইডের Conclusion Speech (নিচে) ব্যবহার করুন।

G. Delivery Tips

- **গুরুত্ব:** চারটি শিক্ষা ধীরে ধীরে, প্রতিটির পর সামান্য বিরতি।
- **ইঙ্গিত:** ১, ২, ৩, ৪ ক্রমে আঙুল।
- **শেষ:** সোজা শ্রোতাদের চোখে তাকিয়ে, হাসিমুখে "ধন্যবাদ"।

🎯 Final Preparation Section

১. ৫ মিনিটের বাংলা Summary

Google Maps আসলে একটি সাধারণ মানচিত্র অ্যাপ নয়। এটি একটি বিশাল distributed platform, যা মানচিত্র দেখানো, জায়গা খোঁজা, রাস্তা বের করা এবং live traffic জানানো, সবই একসঙ্গে করে। এই system-কে প্রথমে চারটি স্তরে বোঝা যায়: ওপরে client অ্যাপ, তারপর API Gateway, তারপর বিভিন্ন service, সবার নিচে data infrastructure।

এই system-এর সমস্যা হলো কোটি কোটি মানুষকে খুব দ্রুত ও নির্ভরযোগ্যভাবে উত্তর দেওয়া, যদিও পৃথিবী প্রতিনিয়ত বদলায়। তাই আমাদের ছয়টি Functional Requirement আছে, যেমন মানচিত্র দেখানো, জায়গা খোঁজা, navigation, live traffic, offline map ও location sharing। আর পাঁচটি Non-Functional লক্ষ্য আছে: বিশাল scale, ১০০ মিলিসেকেন্ডের কম latency, ৯৯.৯৯% availability, data না হারানো ও নিরাপত্তা।

এই লক্ষ্য পূরণে প্রথম প্রয়োজন কার্যকর geospatial data সংরক্ষণ ও খোঁজা। এখানে PostGIS, R-tree, QuadTree, Geohash, H3 ও Redis-এর মতো প্রযুক্তি ব্যবহৃত হয়, যাতে কোটি record না ঘেঁটে শুধু আশপাশের কয়েকটি দেখলেই চলে। Routing-এ road network একটি graph, যেখানে Dijkstra ও A* algorithm সেরা পথ বের করে এবং real-time traffic দিয়ে cost বদলানো হয়। মানচিত্র দেখানো হয় tile দিয়ে, যা CDN-এর মাধ্যমে কাছের server থেকে আসে। জায়গা খোঁজায় full-text ও geospatial search একসঙ্গে চলে এবং ranking ফলাফল সাজায়।

Traffic আসে ব্যবহারকারীর ফোন ও sensor থেকে। ETA অনুমানে ML ব্যবহৃত হয়, যেমন Gradient Boosting, LSTM ও GNN। GPS-এর ভুল কমাতে Kalman filter ও HMM-ভিত্তিক map matching ব্যবহার করা হয়। Offline map, location sharing ও geocoding অতিরিক্ত গুরুত্বপূর্ণ সেবা।

পুরো system চলে cloud-এ: load balancer, Kubernetes, microservices এবং Spanner/Bigtable-এর মতো distributed storage। API Gateway authentication, routing ও rate limiting করে। Caching ও CDN বেশিরভাগ request পথেই শেষ করে। নিরাপত্তায় আছে OAuth 2.0, TLS ও privacy control। Bottleneck, failure ও trade-off-ও আছে, যেমন hot tile, fan-out, circuit breaker ও freshness বনাম cost।

সবশেষে, মানচিত্র ধারাবাহিকভাবে হালনাগাদ হয় data pipeline ও AI/computer vision দিয়ে। শিক্ষাগুলো: cache first, ছোট service, ব্যর্থতার জন্য design, ও data freshness একটি সচেতন সিদ্ধান্ত। মনে রাখবেন, এটি একটি reference architecture, Google-এর প্রকৃত internal design নয়।

---

৪. সবচেয়ে গুরুত্বপূর্ণ ২০টি Technical Term-এর বাংলা অর্থ

1. **Distributed System:** অনেক কম্পিউটারে ভাগ করে চালানো system, যেখানে একটি নষ্ট হলেও বাকিরা চলে।
2. **Geospatial Data:** অবস্থান-সংশ্লিষ্ট (latitude/longitude) তথ্য।
3. **API Gateway:** সব request-এর একক প্রবেশপথ, যা নিরাপত্তা ও routing সামলায়।
4. **Microservices:** একটি বড় অ্যাপকে ছোট স্বাধীন service-এ ভাগ করা।
5. **Latency:** request থেকে উত্তর আসার সময়।
6. **Throughput:** প্রতি সেকেন্ডে কত request সামলানো যায়।
7. **Availability:** system কত শতাংশ সময় চালু থাকে।
8. **Scalability:** চাহিদা বাড়লে সক্ষমতা বাড়ানোর ক্ষমতা।
9. **Spatial Index:** স্থান-তথ্য দ্রুত খোঁজার সূচি (R-tree, QuadTree, Geohash)।
10. **Graph:** node (মোড়) ও edge (রাস্তা) দিয়ে তৈরি কাঠামো।
11. **Dijkstra / A*:** সর্বনিম্ন খরচের পথ বের করার algorithm, A* গন্তব্যমুখী।
12. **Map Tile:** মানচিত্রের ছোট বর্গাকার টুকরো।
13. **CDN:** বিশ্বজুড়ে ছড়ানো server-এর নেটওয়ার্ক, যা কাছের জায়গা থেকে data দেয়।
14. **Cache / TTL:** দ্রুত পুনর্ব্যবহারের জন্য সাময়িক সংরক্ষণ; TTL = কতক্ষণ বৈধ।
15. **ETA:** পৌঁছানোর আনুমানিক সময়।
16. **Map Matching:** GPS বিন্দুকে সবচেয়ে সম্ভাব্য রাস্তায় বসানো।
17. **HMM / Viterbi:** লুকানো অবস্থা অনুমান করার সম্ভাবনা-মডেল ও সবচেয়ে সম্ভাব্য ক্রম বের করার algorithm।
18. **Kubernetes:** container চালানো ও scale করার orchestration ব্যবস্থা।
19. **Spanner / Bigtable:** Google-এর বিতরিত database ও NoSQL storage।
20. **OAuth 2.0:** পাসওয়ার্ড না দিয়ে সীমিত অনুমতি দেওয়ার নিয়ম।

> **নোট:** Final Preparation-এর সেকশনগুলো আপনার তালিকার নম্বর অনুযায়ী (২, ৩, ৫, ৬, ৭) নিচে আছে। ১ ও ৪ নম্বর ওপরে দেওয়া হয়েছে।

২. ১০ মিনিটের বাংলা Presentation Script

(এটি পুরো ২৩ স্লাইড একটানা বলার সংক্ষিপ্ত রূপ। সময় কম থাকলে এটিই ব্যবহার করুন। মোটামুটি ১১০–১২০ শব্দ প্রতি মিনিট ধরে লেখা।)

[স্লাইড ১–২, প্রায় ১ মিনিট]

আসসালামু আলাইকুম স্যার এবং সবাইকে। আমাদের আজকের বিষয় "Inside Google Maps: how a global mapping platform is designed"। আমরা প্রায় সবাই প্রতিদিন Google Maps ব্যবহার করি, কিন্তু এর পেছনে কী ঘটে তা দেখব আজ। শুরুতেই একটা কথা, Google তাদের ভেতরের architecture প্রকাশ করেনি, তাই আমরা যা দেখাব সেটি একটি conceptual reference architecture।

Google Maps আসলে একটি বিশাল distributed platform। ওপরে আছে Android, iOS ও ওয়েব client। সব request প্রথমে যায় API Gateway-তে, যেটি authentication, routing, rate limiting ও caching করে। এর পেছনে আছে চারটি ভাগ: Geospatial Services, Intelligence Services, Real-Time Processing আর Data Infrastructure।

[স্লাইড ৩–৪, প্রায় ১ মিনিট]

সমস্যাটা হলো, শত কোটি মানুষ চায় জায়গা খুঁজতে, রাস্তা চিনতে ও live traffic দেখতে, অথচ পৃথিবী প্রতি মুহূর্তে বদলায়। তাই system-কে ছয়টি কাজ করতে হয়: মানচিত্র দেখানো, search, navigation, live traffic ও ETA, offline map আর location sharing। আর পাঁচটি মান ঠিক রাখতে হয়: scale, ১০০ মিলিসেকেন্ডের কম latency, ৯৯.৯৯% availability অর্থাৎ বছরে প্রায় ৫২ মিনিট বন্ধ, data না হারানো, আর নিরাপত্তা।

[স্লাইড ৫, প্রায় ১ মিনিট]

এবার data। কোটি কোটি জায়গার মধ্যে "আমার কাছের রেস্তোরাঁ" খুঁজতে সব record দেখা অসম্ভব। তাই spatial index ব্যবহার হয়। PostGIS database-কে ভৌগোলিক data বোঝায়, R-tree কাছাকাছি object খোঁজে, QuadTree এলাকাকে চার ভাগে ভাগ করে, Geohash ও H3 এলাকাকে cell-এর কোডে রূপ দেয়, আর Redis ফলাফল cache করে রাখে। এগুলো একসঙ্গে কাজ করতে পারে।

[স্লাইড ৬–৭, প্রায় ১.৫ মিনিট]

Routing-এ রাস্তার নেটওয়ার্ক একটি graph, যেখানে মোড় হলো node আর রাস্তা edge। Dijkstra আর A* algorithm সবচেয়ে কম খরচের পথ বের করে। কিন্তু ছোট পথ মানেই দ্রুত পথ নয়, তাই real-time traffic দিয়ে cost বদলানো হয়।

মানচিত্র দেখানো হয় tile দিয়ে। পুরো পৃথিবী ডাউনলোড না করে শুধু পর্দায় যা দরকার সেই ২৫৬ বাই ২৫৬ পিক্সেলের টুকরোগুলো আনা হয়। Tile দুই ধরনের, raster ছবি আর vector data। এগুলো CDN দিয়ে কাছের server থেকে আসে।

[স্লাইড ৮–১০, প্রায় ১.৫ মিনিট]

জায়গা খোঁজায় full-text আর geospatial search একসঙ্গে চলে, তারপর ranking দূরত্ব, প্রাসঙ্গিকতা ও মান দেখে সাজায়। Traffic আসে sensor, ব্যবহারকারীর রিপোর্ট ও crowdsourcing থেকে, আর মানচিত্রে লাল-হলুদ-সবুজে দেখানো হয়, দরকারে নতুন পথ প্রস্তাব করা হয়।

ETA অনুমানে machine learning ব্যবহৃত হয়। আগের traffic pattern, live traffic, আবহাওয়া ও রাস্তার বৈশিষ্ট্য দিয়ে model train করা হয়। Gradient Boosting, LSTM আর Graph Neural Network এখানে পরিচিত পদ্ধতি। GNN-এর ওপর Google-এর নিজের একটি গবেষণা আছে।

[স্লাইড ১১–১৪, প্রায় ১.৫ মিনিট]

GPS-এর ভুল ৫ থেকে ৩০ মিটার হতে পারে। তাই Kalman filter দিয়ে noise কমানো হয়, আর HMM ও Viterbi দিয়ে অবস্থানকে সবচেয়ে সম্ভাব্য রাস্তায় বসানো হয়, যাকে বলে map matching।

Offline map-এ একটি এলাকা ডাউনলোড করে ফোনে রাখা হয়, আর ইন্টারনেট এলে শুধু পরিবর্তনগুলো sync হয়। Location sharing-এ ফোন নিরাপদভাবে অবস্থান পাঠায়, ব্যবহারকারী ঠিক করে কে দেখবে ও কতক্ষণ। আর geocoding ঠিকানাকে স্থানাঙ্কে, reverse geocoding স্থানাঙ্ককে ঠিকানায় রূপ দেয়।

[স্লাইড ১৫–১৮, প্রায় ১.৫ মিনিট]

এই সব চলে cloud architecture-এ। Load balancer ট্রাফিক ভাগ করে, Kubernetes service চালায় ও scale করে, microservices আলাদা আলাদা কাজ সামলায়, আর Spanner ও Bigtable-এর মতো distributed storage data জমা রাখে। API Gateway প্রবেশপথ, আর service-গুলো REST, gRPC বা Pub/Sub দিয়ে কথা বলে।

Caching ও CDN বেশিরভাগ request পথেই শেষ করে দেয়। TTL ঠিক করে cache কতক্ষণ বৈধ। নিরাপত্তায় আছে OAuth 2.0, TLS এনক্রিপশন ও privacy control, যেখানে ব্যবহারকারী নিজের location history মুছতে পারে।

[স্লাইড ১৯–২৩, প্রায় ১.৫ মিনিট]

কোনো system নিখুঁত নয়। Hot tile, বিশাল graph-এ route search আর live update-এর fan-out হলো চাপের জায়গা। অঞ্চল বন্ধ হলে failover, ধীর service-এ timeout ও circuit breaker, আর ভুল data এলে staged rollout ও rollback কাজ করে। আমরা freshness ও cost, accuracy ও speed, আর consistency ও availability-র মধ্যে আপস করি।

মানচিত্র নিরন্তর হালনাগাদ হয় data pipeline দিয়ে, যেখানে স্যাটেলাইট ছবি, Street View ও ব্যবহারকারীর তথ্য AI ও computer vision দিয়ে বিশ্লেষণ করে যাচাই করা হয়।

শেষে চারটি শিক্ষা: cache first, ছোট স্বাধীন service, ব্যর্থতার জন্য design, আর data freshness একটি সচেতন সিদ্ধান্ত। ধন্যবাদ স্যার, ধন্যবাদ সবাইকে। আমরা প্রশ্নের জন্য প্রস্তুত।

---

৫. System Architecture-এর End-to-End Working Process

ধরা যাক, আপনি খুলনা থেকে ঢাকা যাওয়ার route চাইলেন। ধাপে ধাপে যা ঘটে (reference architecture অনুযায়ী):

ধাপ ১: Client। আপনার ফোনে Maps অ্যাপ। আপনি গন্তব্য লিখলেন।

ধাপ ২: Geocoding। লেখা গন্তব্য "ঢাকা" থেকে স্থানাঙ্ক বের করা হলো। এই কাজে text matching ও spatial index (R-tree/QuadTree) ব্যবহৃত হয় (স্লাইড ১৪, ৫)।

ধাপ ৩: Edge ও API Gateway। Request ইন্টারনেট পেরিয়ে নিকটস্থ edge-এ পৌঁছায়, সেখানে TLS দিয়ে সুরক্ষিত। Gateway token ও API key যাচাই করে, rate limit দেখে, request-কে Routing Service-এ পাঠায় (স্লাইড ১৬, ১৮)।

ধাপ ৪: Load Balancer ও Microservice। Load balancer ফাঁকা একটি Routing Service instance বেছে নেয়। Kubernetes এই instance চালু রাখে (স্লাইড ১৫)।

ধাপ ৫: Data আনা। Routing Service রাস্তার graph ও live traffic তথ্য আনে। জনপ্রিয় data cache (Redis/CDN) থেকে, নতুন বা বিরল data Spanner/Bigtable-এর মতো storage থেকে (স্লাইড ১৭, ১৫)।

ধাপ ৬: Route হিসাব। Dijkstra/A* (ও উন্নত পদ্ধতি) দিয়ে প্রতিটি edge-এর সময়-খরচের ভিত্তিতে সেরা পথ বের হয় (স্লাইড ৬)।

ধাপ ৭: ETA। ML model ঐতিহাসিক ও live data দিয়ে পৌঁছানোর সময় অনুমান করে (স্লাইড ১০)।

ধাপ ৮: উত্তর ও map rendering। উত্তর ফোনে আসে। রাস্তা আঁকার জন্য ফোন CDN থেকে vector/raster tile আনে (স্লাইড ৭)।

ধাপ ৯: চলার সময়। GPS data থেকে Kalman filter ও map matching দিয়ে আপনার নীল বিন্দু রাস্তায় বসে (স্লাইড ১১)। আপনার ফোনের গতি-তথ্য (বেনামী ও সমষ্টিগত ভাবে) traffic আপডেটে সাহায্য করে (স্লাইড ৯)।

ধাপ ১০: পুনর্বিবেচনা। সামনে জ্যাম বা দুর্ঘটনা হলে route পুনরায় হিসাব হয় ও নতুন ETA আসে।

পেছনে, ধারাবাহিকভাবে: স্যাটেলাইট, Street View, ব্যবহারকারীর তথ্য থেকে data pipeline ও AI মানচিত্রকে হালনাগাদ রাখে (স্লাইড ২০–২১)। আর পুরো সময় monitoring, failover ও security কাজ করে (স্লাইড ১৮–১৯)।

সংক্ষেপে: Client → Edge/Gateway → Load Balancer → Microservice → Cache/Database → Algorithm/ML → Response → Client।

---

৩. Google Maps System Design: ৩০টি সম্ভাব্য Viva Question ও Answer

সাধারণ ও Architecture

1. **Google Maps-এর overall architecture সংক্ষেপে বলুন।** Client → API Gateway/Edge → চারটি service domain (Geospatial, Intelligence, Real-Time, Data Infrastructure)। এটি একটি reference architecture।
2. **এটি কি Google-এর আসল architecture?** না। Google-এর ভেতরের design প্রকাশিত নয়। এটি প্রচলিত ও গবেষণা-সমর্থিত conceptual architecture।
3. **Functional ও Non-Functional Requirement-এর পার্থক্য?** Functional = system কী করবে। Non-functional = কতটা ভালোভাবে করবে (গতি, নির্ভরযোগ্যতা, নিরাপত্তা)।

4. **Distributed system কেন দরকার?** একটি মেশিন এত চাপ সামলাতে পারে না, আর সেটি নষ্ট হলে সব বন্ধ। বিতরিত system-এ কাজ ভাগ হয় ও ব্যর্থতা সহনীয় হয়।
5. **Monolith বনাম Microservices?** Monolith একটি বড় প্রোগ্রাম, সহজ কিন্তু scale ও পরিবর্তন কঠিন। Microservices ছোট স্বাধীন service, আলাদা scale ও deploy করা যায়, কিন্তু জটিলতা বেশি।
6. **API Gateway-র কাজ কী?** Authentication, routing, rate limiting, monitoring এক জায়গায়।

Data ও Indexing

7. **Spatial index কেন দরকার?** Full scan এড়িয়ে শুধু আশপাশের কয়েকটি record দেখে দ্রুত উত্তর পেতে।
8. **R-tree কীভাবে কাজ করে?** কাছাকাছি object-কে বাক্সে গুচ্ছ করে, বাক্সগুলোকে আবার বড় বাক্সে। query-র সঙ্গে যে বাক্স মেলে না তা বাদ।
9. **QuadTree ও R-tree-এর পার্থক্য?** QuadTree স্থানকে চার ভাগে ভাগ করে (space partitioning), R-tree object গুচ্ছ করে।
10. **Geohash কী ও সীমাবদ্ধতা?** Coordinate-কে স্ট্রিং কোডে রূপ দেয়, কাছাকাছি জায়গার prefix মেলে। সীমানার দুই পাশে কাছাকাছি জায়গার কোড আলাদা হতে পারে, তাই প্রতিবেশী cell-ও দেখতে হয়।
11. **Geohash বনাম H3?** Geohash চতুর্ভুজ, H3 ষড়ভুজ। ষড়ভুজে সব প্রতিবেশী সমদূরত্বে, তাই aggregation ভালো।
12. **PostGIS কেন?** সাধারণ database দূরত্ব বা সীমানা বোঝে না। PostGIS geometry type ও spatial function দেয়।

Algorithm ও ML

13. **Dijkstra কীভাবে কাজ করে?** শুরুর node থেকে ধাপে ধাপে সবচেয়ে কম দূরত্বের node বেছে প্রতিবেশীদের দূরত্ব হালনাগাদ করে, গন্তব্য পাওয়া পর্যন্ত।
14. **A* কেন Dijkstra-র চেয়ে ভালো?** Heuristic (গন্তব্যের আনুমানিক দূরত্ব) ব্যবহার করে গন্তব্যমুখী হয়, কম node দেখে।
15. **সবচেয়ে ছোট পথ কি সবসময় দ্রুততম?** না। জ্যাম, রাস্তা বন্ধ ও গতিসীমা বিবেচনা করতে সময়কে cost ধরা হয়।
16. **ETA কীভাবে অনুমান করা হয়?** ঐতিহাসিক ও live traffic, আবহাওয়া, রাস্তার বৈশিষ্ট্য দিয়ে ML model (GBDT, LSTM, GNN) train করে।
17. **GNN কেন ব্যবহার হয়?** রাস্তা নিজেই graph, আর প্রতিবেশী রাস্তা একে অপরকে প্রভাবিত করে। GNN এই সম্পর্ক ধরে।
18. **Map matching কেন দরকার?** GPS ভুল হয় (৫–৩০ মিটার)। তাই বিন্দুকে সবচেয়ে সম্ভাব্য রাস্তায় বসাতে হয়।
19. **HMM ও Viterbi কী?** HMM-এ আসল রাস্তা লুকানো, আমরা GPS মাপ দেখি। Viterbi সবচেয়ে সম্ভাব্য রাস্তার ক্রম দক্ষভাবে বের করে।
20. **Kalman Filter কী করে?** পূর্বানুমান ও নতুন মাপ মিলিয়ে noise কমিয়ে মসৃণ অবস্থান দেয়।

Performance ও Caching

21. **Map tile কেন?** পুরো মানচিত্র একসঙ্গে ডাউনলোড অসম্ভব। শুধু পর্দার tile আনলে data কম ও cache সহজ।
22. **Raster বনাম Vector tile?** Raster তৈরি ছবি, Vector জ্যামিতিক data যা ফোন আঁকে। Vector ছোট ও স্টাইলযোগ্য।
23. **CDN কীভাবে সাহায্য করে?** কাছের edge server থেকে data দেয়, তাই latency ও মূল server-এর চাপ কমে।
24. **Cache invalidation কঠিন কেন?** Data বদলালে কোন কপি বাতিল হবে তা ঠিক করা কঠিন। বেশি রাখলে পুরোনো, কম রাখলে সুবিধা কম।

Scale, Reliability ও Security

25. **Horizontal ও Vertical scaling?** Horizontal = আরও মেশিন, Vertical = একটি মেশিন শক্তিশালী। বিশাল system-এ horizontal।
26. **Kubernetes কী ও কেন?** Container চালানো ও পরিচালনার orchestration। Self-healing ও autoscaling দেয়।
27. **Spanner বনাম Bigtable?** Spanner বিশ্বব্যাপী relational, শক্ত consistency। Bigtable NoSQL, বিশাল ও দ্রুত।
28. **Circuit breaker কী?** ব্যর্থ service-এ সাময়িক request বন্ধ রেখে ব্যর্থতা ছড়ানো ঠেকায়।
29. **Map পড়ায় Availability কেন Consistency-র আগে?** সামান্য পুরোনো মানচিত্র ব্যবহারযোগ্য, কিন্তু মানচিত্র না পাওয়া নয় (CAP trade-off)।
30. **Location privacy কীভাবে রক্ষা হয়?** সম্মতি, নিয়ন্ত্রণ (কে দেখবে, কতক্ষণ), মোছার সুযোগ, TLS/encryption, আর data minimization।

---

৬. "Explain the overall architecture of Google Maps", এই প্রশ্নের উত্তর

(প্রায় ১.৫–২ মিনিটের উত্তর। এটি মুখস্থ নয়, কাঠামোটি মনে রাখুন: ওপর থেকে নিচে চার ধাপ।)

> স্যার, প্রথমেই বলে নিই, Google তাদের ভেতরের architecture প্রকাশ করেনি। আমি একটি conceptual reference architecture ব্যাখ্যা করব, যা প্রকাশিত গবেষণা ও প্রচলিত system design নীতির ওপর ভিত্তি করে তৈরি।
>
> আমি পুরো system-কে ওপর থেকে নিচে চার ধাপে দেখি। সবার ওপরে **Client স্তর**: Android, iOS ও ওয়েব। এখান থেকে request আসে। তারপর **Edge ও API Gateway স্তর**। Request এখানে প্রথম পৌঁছায়। এই স্তর authentication, routing, rate limiting ও caching করে। CDN ও load balancer-ও এই স্তরের অংশ, যারা কাছের server থেকে উত্তর দেয় ও ট্রাফিক ভাগ করে।
>
> তৃতীয় স্তরে আছে **microservices**, যেখানে প্রতিটি service একটি নির্দিষ্ট কাজ করে: Map ও Tile service মানচিত্র দেয়, Places ও Search service জায়গা খোঁজে, Routing ও Traffic service পথ ও ETA বের করে, আর Geocoding service ঠিকানা ও স্থানাঙ্কের রূপান্তর করে। এই service-গুলো Kubernetes-এর মতো ব্যবস্থায় চলে ও স্বাধীনভাবে scale করে। এদের পেছনে আছে বুদ্ধিমত্তার অংশ: graph algorithm দিয়ে route, machine learning দিয়ে ETA, আর GPS-এর জন্য map matching।
>
> সবার নিচে **Data স্তর**: Spanner ও Bigtable-এর মতো distributed storage, geospatial index, আর Redis-এর মতো cache। এই স্তরে map data, places data, traffic data ও user data থাকে, একাধিক অঞ্চলে কপিসহ।
>
> এর পাশাপাশি দুটি ধারা সব স্তর জুড়ে চলে। একটি হলো **data pipeline ও AI**, যা স্যাটেলাইট ছবি, Street View ও ব্যবহারকারীর তথ্য থেকে মানচিত্রকে হালনাগাদ রাখে। অন্যটি হলো **cross-cutting বিষয়**: নিরাপত্তা (OAuth, TLS, privacy), observability (log, metrics, tracing) আর ব্যর্থতা-সহনশীলতা (failover, circuit breaker)।
>
> সব মিলিয়ে লক্ষ্য থাকে কম latency, বেশি availability আর বিশ্বব্যাপী scale, আর তাই নকশার মূল নীতি: cache first, ছোট স্বাধীন service, ব্যর্থতার জন্য design, আর data freshness-কে সচেতন সিদ্ধান্ত হিসেবে দেখা।

মনে রাখার কৌশল: "**ক্লায়েন্ট → গেট → সার্ভিস → ডাটা**, পাশে **AI পাইপলাইন** আর **নিরাপত্তা/পর্যবেক্ষণ**।"

---

৭. Presentation-এর সমাপনী (Conclusion) Speech

> সম্মানিত স্যার এবং আমার প্রিয় সহপাঠীরা, আজ আমরা একটি পরিচিত অ্যাপের ভেতরে উঁকি দিয়েছি।
>
> আমরা দেখলাম, যে মানচিত্র আমরা প্রতিদিন কয়েক সেকেন্ডে খুলে ফেলি, তার পেছনে আছে একটি বিশাল বিতরিত ব্যবস্থা। সেখানে geospatial index কোটি কোটি জায়গা থেকে মুহূর্তে আমাদের কাছেরটি খুঁজে দেয়, graph algorithm ও machine learning মিলে পথ ও সময় বের করে, tile ও CDN মানচিত্রকে মসৃণ রাখে, আর data pipeline ও AI পৃথিবীর পরিবর্তনের সঙ্গে মানচিত্রকে নিরন্তর হালনাগাদ রাখে।
>
> আমরা আরও দেখলাম, কোনো বড় system নিখুঁত নয়। সেখানে চাপের জায়গা আছে, ব্যর্থতা আছে, আর আছে আপস: তাজা data বনাম খরচ, নিখুঁত বনাম দ্রুত। ভালো system design মানে সব সমস্যা মুছে ফেলা নয়, বরং বুঝে-শুনে সঠিক সিদ্ধান্ত নেওয়া। আমরা যে চারটি শিক্ষা পেলাম, cache first, ছোট স্বাধীন service, ব্যর্থতার জন্য design এবং data freshness-কে সচেতন সিদ্ধান্ত হিসেবে দেখা, এগুলো শুধু Google Maps-এর জন্য নয়, যেকোনো বড় system-এর জন্যই প্রযোজ্য।
>
> আমরা মনে করিয়ে দিই, আমাদের উপস্থাপিত architecture একটি conceptual reference, Google-এর প্রকৃত ভেতরের নকশা নয়। তবু আমরা বিশ্বাস করি, এটি আমাদের বড় system কীভাবে ভাবতে হয় তা শিখিয়েছে।
>
> আপনাদের ধৈর্য ধরে শোনার জন্য আন্তরিক ধন্যবাদ। এখন আপনাদের যেকোনো প্রশ্ন বা মতামতের জন্য আমরা প্রস্তুত।

🔖 শেষ মুহূর্তের পরামর্শ (পরীক্ষার আগের রাতে)

1. প্রথমে এই ৪টি জিনিস মনে রাখুন: **Disclaimer** (reference architecture), **চারটি Layer** (Client → Gateway → Service → Data), **পাঁচটি Non-Functional লক্ষ্য**, আর **চারটি Key Lesson**।
2. ৩টি জিনিস ভালোভাবে বুঝে নিন, কারণ শিক্ষক এগুলো থেকেই প্রশ্ন করবেন: **Dijkstra/A***, **Spatial Index (R-tree/Geohash)**, আর **Caching/CDN**।
3. যে প্রশ্নের উত্তর জানেন না, বলুন: "স্যার, এটি আমাদের আলোচনার বাইরে, তবে আমার বোঝা অনুযায়ী…" তারপর সংশ্লিষ্ট মূল ধারণা থেকে উত্তর দিন।
4. **Notes ভুল (স্লাইড ৩, ৪, ১৯, ২২)**: ওই স্লাইডের Notes পড়বেন না, এই গাইডের Speech ব্যবহার করুন।
5. আত্মবিশ্বাসের জন্য: ধীরে বলুন, প্রতিটি নতুন term-এর পর এক বাক্যে অর্থ বলুন।

শুভকামনা রইল। 🌟